StandardHaloModelsPP.jl

Thomas (Tom) C. Gorordo

January 1, 1980

Contents

Contents ii

I Introduction - Galactic-Frame Velocity Distributions 1
1 Presets 4
2 The Standard Halo Model 5
3 Anisotropic Refinement - the SHM++ 7
4 Measurement Uncertainties 9

II Galactic & Solar Ephemerides - the Earth-Frame Distributions 10
5 Ephemerides 12
6 Lab-Frame Distributions 18

III Velocity Distribution Integrals 19
7 Marginalized Distributions 21
8 Mean Speed and Reciprocal 23
9 Arbitrary Convolutions 25
10 Integration Problems 27
10.1 Sweeps . . . 28
10.2 Building Your Own Integrals . . . 29

IV Performance Tips 30
11 Evaluating a Distribution at Many Points 31
12 Uncertainties Cost Time 32
13 Analytic Fast Paths 33
14 Choosing Between `velint` and `speedint` 34
15 Tolerances 35
16 Many Integrals: Sweeps and Threads 36

V Reference 37
17 Reference 38
17.1 Bibliography . . . 38
17.2 API . . . 39

Part I

Introduction - Galactic-Frame Velocity Distributions

StandardHaloModelsPP.jl implements Dark Matter velocity distributions, and integrals of them, for the Standard Halo Model (**SHM**) and an extension of the distribution, the **SHM++**. The SHM is an isotropic, truncated Maxwellian [1]; by default its parameters follow the recommended conventions for reporting direct-detection results [2]. The SHM++ adds an anisotropic component to better model local measurements, accounting for the "Gaia Sausage" [3, 4], as defined by:

[5]: N. Wyn Evans, Ciaran A.J. O'Hare, Christopher McCabe, *Phys. Rev. D.* **99**, 023012 (2019) (arXiv:1810.11468)

A HaloModel struct encodes the SHM++ parameterization, and Presets are provided for common choices. This page covers the galactic-frame distributions; their lab-frame counterparts and integrals follow.

StandardHaloModelsPP.HaloModel – Type.

```
HaloModel(ρ₀::Union{Unitful.Pressure,Unitful.Density}, v₀::Unitful.Velocity,
↪  vₑ::Unitful.Velocity, β::Real, η::Real[, VLSR::Unitful.Velocity])
```

An SHM++-parameterized halo model, per the definition in N. Wyn Evans, Ciaran A.J. O'Hare, Christopher McCabe, *Phys. Rev. D.* **99**, 023012 (2019) (arXiv).

- `localdensity`: `ρ₀`, local DM mass density: with mass in energy units (e.g. `u"GeV/cm^3"`, hence a `Unitful` pressure), or as a mass density (e.g. `UnitfulAstro`: `u"Msun/pc^3"`); units are stored as given.
- `circspeed`: `v₀`, circular speed of the flat asymptote of the galactic rotation curve, which sets the DM velocity dispersion $\sigma = v_0/\sqrt{2}$.
- `escspeed`: `vₑ`, galactic escape speed.
- `anisotropy`: `β`, Gaia sausage anisotropy, `0 ≤ β < 1`.
- `sausagefrac`: `η`, Gaia sausage prominence, `0 ≤ η ≤ 1`.
- `lsrspeed`: `VLSR`, speed of the Local Standard of Rest, which sets the boost to the lab frame. By default (omitted, or `nothing`) it is `v₀`: the SHM's isothermal halo has a flat rotation curve with circular speed $v_0 = \sqrt{2}\sigma$, so the two coincide [5]. Give it to boost with a different speed than the dispersion implies. `m.VLSR` is always the speed in use, even when tied to `v₀`.

source

StandardHaloModelsPP.HaloModel – Method.

```
HaloModel(; ρ₀, v₀, vₑ, β = 0, η = 0, VLSR = nothing)
HaloModel(m::HaloModel; kwargs...)
```

Keyword forms of the HaloModel constructor. Each parameter may be named by its field name or a symbol shorthand (`v₀` or `circspeed`, `vₑ` or `escspeed`, ...).

Also a copying form which copies `m` with the given parameters changed, e.g. `HaloModel(SHMpp; v₀ = 220u"km/s")`. Unchanged `Measurement` parameters are carried over as the same objects to maintain correlations.

This also allows e.g. Accessors.jl to work automatically on symbol shorthands: `@set SHMpp.v₀ = 220u"km/s"`.

source

Every function takes a model as an optional first argument, defaulting to the SHM (e.g. `fχ(v) == fχ(SHM, v)`), and every lab-frame function an optional time after it, defaulting to T0 (June 1st.).

Physical Units

Quantities carry units using Unitful.jl, as re-exported by NaturallyUnitful.jl: densities in e.g. `u"GeV/cm^3"`, speeds in e.g. `u"km/s"`. Any compatible units are accepted, so e.g. you may use UnitfulAstro.jl.

Chapter 1

Presets

Preset	ρ_0 (GeV cm^{-3})	v_0 (km s^{-1})	v_{esc} (km s^{-1})	β	η	Use
`SHM`	0.3	238	544	0	0	The SHM of [2], and default model
`SHMlegacy`	0.3	220	544	0	0	A historical SHM, for comparison with older results
`SHMnoesc`	0.3	238	∞	0	0	The SHM without escape-speed truncation
`SHMerr`	0.3	238 ± 1.5	544 ± 33	0	0	The SHM with measurement uncertainties, for sensitivity studies
`SHMpp`	0.55 ± 0.17	233 ± 3	528 ± 25	0.9 ± 0.05	0.2 ± 0.1	The SHM++ of [5]

Lab-frame boosts (see more on Ephemerides) use the LSR speed $v_{\mathrm{LSR}} = v_0$ by default. Other models can be built with the `HaloModel` constructors: e.g. `HaloModel(SHM; vₑ = 500u"km/s")`, or with a different LSR speed, `HaloModel(SHMpp; VLSR = 238u"km/s")`.

Chapter 2

The Standard Halo Model

The SHM presets follow the conventions of [2, Table 1], with the historical $v_0 = 220$ km/s variant of [5, Table I], an untruncated variant, and a variant with measurement uncertainties:

`StandardHaloModelsPP.SHM` – Constant.

The default `HaloModel` for the SHM, following the recommended conventions for reporting direct-detection results in D. Baxter et al., *Eur. Phys. J. C* **81**, 907 (2021) (arXiv)

SHM		
Local DM density	ρ_0	0.3 GeV cm^{-3}
Circular rotation speed	v_0	238 km s^{-1}
Escape speed	v_{esc}	544 km s^{-1}
Velocity distribution	$f_R(\mathbf{v})$	Gaussian

(additional SHM++ parameters are zeroed.) [2, Table 1]

`v₀`, and so the LSR speed of the lab-frame boost, is the reference ephemeris' `VLSR` (its central value). For the pre-2021 `v₀ = 220` km/s convention see `SHMlegacy`.

source

`StandardHaloModelsPP.SHMlegacy` – Constant.

A historical SHM, with `v₀ = 220` km/s with lab-frame boosts commensurate (otherwise as `SHM`), for comparison with results that predate [2]; see e.g. [1] and [6, Table 1].

source

`StandardHaloModelsPP.SHMnoesc` – Constant.

An untruncated variant of `SHM`: `vₑ = Inf`, so the Maxwellian has no escape-speed cutoff and normalizes to unity over all velocities.

source

`StandardHaloModelsPP.SHMerr` – Constant.

`SHM` with measurement uncertainties on its two characteristic speeds *included*, for sensitivity studies of Halo Model conventions and uncertainties.

- `v₀`: using uncertainty quoted but not used in [2, Sec. 3.1.5].

SHMerr		
Local DM density	ρ_0	0.3 GeV cm^{-3} (fixed)
Circular rotation speed	v_0	238.0 ± 1.5 km s^{-1}
Escape speed	v_{esc}	544 ± 33 km s^{-1}

- `vₑ`: using the RAVE 90% confidence interval $498 < v_{\text{esc}} < 608$ km/s of [7] (median 544 km/s, agreeing with the `SHM` convention), symmetrized and converted to 1σ: $(608-498)/(2\times1.645) \approx 33$ km/s.
- `ρ₀` is a fixed value rather than a measurement with uncertainty, as it is not a directly measured quantity; set another value (usually in the range 0.2-0.6 GeV/cm³) by creating your own `HaloModel` from this one via `HaloModel`.
- $v_{\text{LSR}} = v_0$ as `Measurements.jl` objects so the dispersion and the boost are correctly correlated.

Note

[2, Sec. 3.1] recommends against attaching uncertainties to the SHM parameters when reporting results: they "do not accurately reflect the uncertainties in the dark matter halo, nor do they represent likelihood distributions that can be meaningfully profiled over". `SHM` therefore stays exact (and is the default model); use `SHMerr` for sensitivity studies, but not for reporting limits. Propagating Measurements uncertainties also has a significant performance impact on integrals.

source

Its velocity distribution is a Maxwellian truncated at the escape speed [5, Eqs. (1, 2)]:

`StandardHaloModelsPP.fR` – Method.

```
fR([m::HaloModel = SHM,] v::AbstractVector{<:Unitful.Velocity})
fR([m::HaloModel = SHM,] v::Unitful.Velocity)
fR(m::HaloModel) -> v -> fR(m, v)
```

Galactic-frame Maxwellian component of f_χ.

$$f_R(\mathbf{v}) = \frac{1}{(2\pi\sigma_v^2)^{3/2} N_{R,\text{esc}}} \exp\left(-\frac{|\mathbf{v}|^2}{2\sigma_v^2}\right) \Theta(v_{\text{esc}} - |\mathbf{v}|)$$

$$N_{R,\text{esc}} = \text{erf}\left(\frac{v_{\text{esc}}}{\sqrt{2}\sigma_v}\right) - \sqrt{\frac{2}{\pi}} \frac{v_{\text{esc}}}{\sigma_v} \exp\left(-\frac{v_{\text{esc}}^2}{2\sigma_v^2}\right)$$

with $\sigma_v = v_0/\sqrt{2}$. Isotropic, so `v` may be given as a scalar speed.

For `fR(m)` see "Repeated evaluation" under `fχ`.

source

Chapter 3

Anisotropic Refinement - the SHM++

The SHM++ adds two parameters for the so-called "Gaia Sausage", its anisotropy β and its fraction η of the local DM [5, Table I]:

`StandardHaloModelsPP.SHMpp` – Constant.

The default `HaloModel` for the SHM++.

SHM++		
Local DM density	ρ_0	0.55 ± 0.17 GeV cm^{-3}
Circular rotation speed	v_0	233 ± 3 km s^{-1}
Escape speed	v_{esc}	528 ± 25 km s^{-1}
Sausage anisotropy	β	0.9 ± 0.05
Sausage fraction	η	0.2 ± 0.1
Velocity distribution	$f_\chi(\mathbf{v})$	Gaussian+Sausage

[6, Table 1]

As in the defining SHM++ paper, the lab-frame boost uses $v_{\text{LSR}} = v_0 = 233 \pm 3$ km/s, with the boost and the dispersion correlated. Create your own modified copy if you want a SHM++ halo model with Baxter [2] parameters instead.

You may want to strip Measurement uncertainties via `Measurements.value(m)` if you are not interested in sensitivity questions wrt. the Gaia sausage and just want to use this as a halo model.

source

Its distribution mixes the round SHM component f_R with the anisotropic sausage component f_S [5, Eq. (3)]:

`StandardHaloModelsPP.fχ` – Method.

```
fχ([m::HaloModel = SHM,] v::AbstractVector{<:Unitful.Velocity})
fχ(m::HaloModel) -> v -> fχ(m, v)
```

Galactic-frame SHM++ velocity distribution: a mixture of the round `fR` and anisotropic `fS` components.

$$f_\chi(\mathbf{v}) = (1-\eta) f_R(\mathbf{v}) + \eta f_S(\mathbf{v})$$

Repeated evaluation

Each `fχ(m, v)` call recomputes the model's normalizations (`erf`, `erfi` and `exp` of the parameters), which for a `Measurement` model costs several times the evaluation itself.

The one-argument form `f = fχ(m)` computes them once and returns a callable `f(v)`, to use in hot loops, closures or integrands. Broadcasting over velocities with a single model, as in `fχ.(m, vs)` or `fχ.(vs)`, does this automatically.

The same applies to `fR`, `fS`, `fRlab` and `fχlab`.

source

with the sausage [5, Eqs. (5-7)]:

`StandardHaloModelsPP.fS` – Method.

```
fS([m::HaloModel = SHM,] v::AbstractVector{<:Unitful.Velocity})
fS(m::HaloModel) -> v -> fS(m, v)
```

Galactic-frame anisotropic (Gaia sausage) component of f_χ.

$$f_S(\mathbf{v}) = \frac{1}{(2\pi)^{3/2}\sigma_r\sigma_\theta^2 N_{S,\mathrm{esc}}} \exp\left(-\frac{v_r^2}{2\sigma_r^2} - \frac{v_\theta^2}{2\sigma_\theta^2} - \frac{v_\phi^2}{2\sigma_\phi^2}\right) \Theta(v_{\mathrm{esc}} - |\mathbf{v}|)$$

$$N_{S,\mathrm{esc}} = \mathrm{erf}\left(\frac{v_{\mathrm{esc}}}{\sqrt{2}\sigma_r}\right) - \sqrt{\frac{1-\beta}{\beta}} \exp\left(-\frac{v_{\mathrm{esc}}^2}{2\sigma_\theta^2}\right) \mathrm{erfi}\left(\frac{v_{\mathrm{esc}}}{\sqrt{2}\sigma_r}\sqrt{\frac{\beta}{1-\beta}}\right)$$

Anisotropic, so v must be a vector; `v[1]` is the radial (sausage) axis v_r.

For `fS(m)` see "Repeated evaluation" under `fχ`.

source

The Galactic-Frame Distribution

`fχ` is the full galactic-frame distribution, and usually what you want to use. To evaluate it at many velocities, see Performance Tips.

Chapter 4

Measurement Uncertainties

Parameters with measurement uncertainties use Measurements.jl, as do the ephemeris constants (see Ephemerides).

- Pointwise functions (`fχ`, `fχlab`, ...) follow the model: a model carrying `Measurements` gives `Measurement` results, and an exact one plain numbers (with the central-value ephemeris).
- Integrals (`fχspeed`, `g`, `velint`, `HaloIntegralProblem`, ...) evaluate central values by default, since propagating through a quadrature is typically 10-130x slower; pass `uncertainty = true` to propagate.
- `SHM` is exact, following the reporting conventions of [2]; `SHMerr` attaches measurement uncertainties to its speeds for sensitivity studies (not for reported limits), and `SHMpp` carries the uncertainties of the SHM++.

The central-value model of any `HaloModel` is `value(m)` (from Measurements.jl), which evaluates as fast as an exact model:

`Measurements.value` – Method.

```
Measurements.value(m::HaloModel)
```

Copy `m` with every `Measurement` parameter replaced by its central value.

source

Part II

Galactic & Solar Ephemerides - the Earth-Frame Distributions

The Earth-frame distributions follow from the galactic-frame ones by a boost with the Earth's galactic velocity as a composition of the Earth-Sun and Sun-(Sag. A⋆) orbits.

`StandardHaloModelsPP.jl` provides a simple model of that motion in which the Sun's ephemeris follows the conventions of [2], with its peculiar velocity from [8] and its position from [9], and the Earth's orbit follows the conventions of [10, 11].

Chapter 5

Ephemerides

The Sun's galactic velocity $\vec{v}_\odot = \vec{v}_{\text{LSR}} + \vec{v}_{\text{PEC}}$ is that of the Local Standard of Rest (LSR), the circular orbit about the Galactic Center at the Sun's position, plus the Sun's peculiar velocity relative to it. Schematically (not to scale):

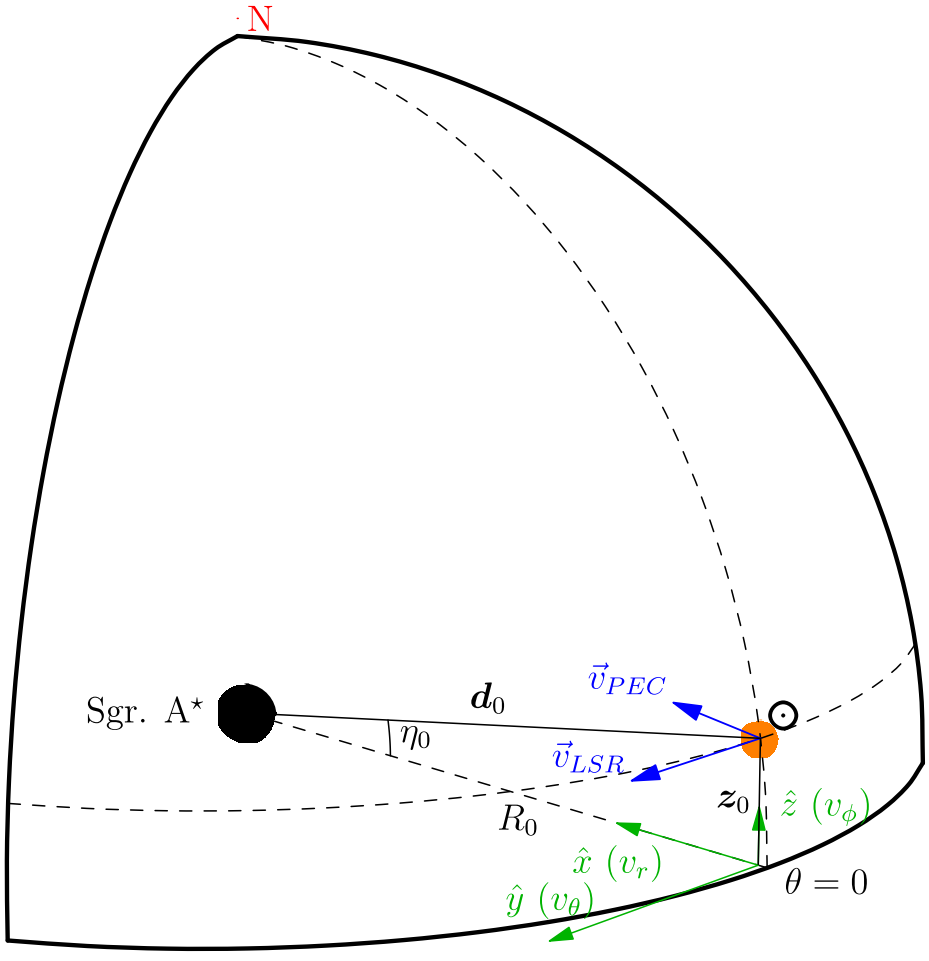


Figure 5.1: The Sun's galactocentric position, the Galactic-Rectangular axes (along which the halo velocity components are taken), and the LSR and peculiar velocities of the Sun.

(Fig. gal.asy source)

Galactic-Rectangular coordinates (green, centered in the plane below the Sun) are defined with $\hat{x}$ pointing from the Sun to Sgr A*, $\hat{y}$ along the disk's rotation (clockwise seen from the North Galactic Pole), and $\hat{z}$ to the North Galactic Pole [10] [12, Sec. 10.3.1].

The DM halo velocity distribution is usually expressed in Galactocentric-Spherical coordinates [5]. Its components at the sun are taken along the Galactic-Rectangular axes [1], and at the location of the solar system $(v_r, v_\theta, v_\phi) \equiv (v_x, v_y, v_z)$ (labeled in the figure).

The lab-frame distribution is related by a Galilean boost in that frame [10]: a DM velocity $\vec{v}$ in the lab is $\vec{v} + \vec{v}_E$ in the galaxy; $f_{\text{lab}}(\vec{v}, t) = f(\vec{v} + \vec{v}_E(t))$.

Position of the Sun:

`StandardHaloModelsPP.d0` – Constant.

Distance of the sun from the Galactic Center (Sgr. A*).

Value: $d_0 =$ 8.275 ± 0.034 kpc.

R. Abuter and others (GRAVITY collaboration). Astron. Astrophys. 647, A59 (2021): $8275 \pm 9_{\text{stat}} \pm 33_{\text{sys}}$ pc, combined in quadrature. This is the distance underlying VLSR in [2]; it supersedes the 8.122 ± 0.031 kpc of [13] used in the defining SHM++ paper.

source

`StandardHaloModelsPP.z0` – Constant.

Normal distance of the sun above the Galactic Plane.

Value: $z_0 =$ 25.0 ± 5.0 pc

J. Bland-Hawthorn and O. Gerhard, Ann. Rev. Astron. Astrophys. 54, 529 (2016), adopting the SDSS estimate of M. Jurić and others (SDSS collaboration), Astrophys. J. 673, 864 (2008).

source

`StandardHaloModelsPP.θ0` – Constant.

Azimuthal angle of the solar system in Galactocentric Spherical Coordinates.

Value: $\theta_0 =$ 0.0

(Zero by definition.)

source

`StandardHaloModelsPP.η0` – Constant.

Altitude (angle) of the sun above the Galactic Plane.

Value: $\eta_0 = \sin^{-1} z_0/d_0 =$ 0.00302 ± 0.0006 (radians)

source

`StandardHaloModelsPP.ϕ0` – Constant.

Polar angle of the sun from the galactic north zenith.

Value: $\phi_0 = \frac{\pi}{2} - \eta_0 =$ 1.56778 ± 0.0006 (radians)

source

[1] The angle of the sun above the galactic plane is small and can be neglected – η0 is *extremely* exaggerated in the figure.

`StandardHaloModelsPP.R0` – Constant.

Radial (in-galactic-plane/cylindrical) distance of the sun from the Galactic Center (Sgr. A*).

Value: $R_0 = d_0 \cos \eta_0 =$ 8.275 ± 0.034 kpc

source

Velocity of the Sun:

`StandardHaloModelsPP.VLSR` – Constant.

Reference circular speed of the Local Standard of Rest (LSR), used by ephemeris functions when no model is given.

Value: 238.0 ± 1.5 km s^-1

D. Baxter and others, Eur. Phys. J. C 81, 907 (2021); Sec. 3.1.5.

Note

A `HaloModel` carries its own LSR speed `m.VLSR`, which by default is its circular speed `v₀`. The model-aware `vE(m, t)` and every lab-frame distribution and integral use that parameter when a model is available, not this constant. It equals this one for `SHM` and `SHMerr`, but is 233 km/s for `SHMpp` and 220 km/s for `SHMlegacy`, as in their defining papers.

The derived reference constants `vSUN`, `T0`, VEavg and Tavg use this value whatever the model. `T0` in particular is a fixed date.

source

`StandardHaloModelsPP.vLSR` – Constant.

Circular velocity (vector) of the Local Standard of Rest (LSR) in galactic rectangular coordinates.

Value: $v_{\text{LSR}} = (v_x, v_y, v_z) =$ (0.0 ± 0.0 km s^-1, 238.0 ± 1.5 km s^-1, 0.0 ± 0.0 km s^-1)

source

`StandardHaloModelsPP.vPEC` – Constant.

Peculiar velocity of the sun in the Local Standard of Rest (LSR) in galactic rectangular coordinates.

Value: $v_{\text{PEC}} = (U, V, W) =$ (11.1 ± 1.5 km s^-1, 12.2 ± 2.0 km s^-1, 7.3 ± 1.0 km s^-1)

R. Schönrich, J. Binney, W. Dehnen, Mon. Not. Roy. Astron. Soc. 403, 1829 (2010), with the (statistical and systematic) uncertainties as adopted by the SHM++.

source

`StandardHaloModelsPP.vSUN` – Constant.

Velocity of the sun in the Galactic frame, using the reference-value `VLSR`. The halo's Galactocentric-Spherical velocity components are taken along the Galactic-Rectangular axes at the Sun [5].

Value: $v_\odot = (v_x, v_y, v_z) = (v_r, v_\theta, v_\phi) =$ (11.1 ± 1.5 km s^-1, 250.2 ± 2.5 km s^-1, 7.3 ± 1.0 km s^-1)

source

`StandardHaloModelsPP.VSUN` – Constant.

Speed of the sun in the galactic frame. A good ballpark typical DM velocity ignoring Earth ephemeris.

source

In most halo models the DM (isothermal) distribution is assumed to be stationary with respect to the galaxy, so it is the motion of the solar system through the galaxy that sets v_0, the DM velocity dispersion [5] – i.e. the LSR speed is essentially that parameter of a halo model. Each `HaloModel` therefore carries its own LSR speed `m.VLSR`, by default equivalent to its `v₀`, and ephemeris functions are model-aware; i.e. `vE(m, t)`, the lab-frame distributions and the integrals all use it for their boost to the lab. The constants here are a reference ephemeris, with `VLSR` from [2] (that of `SHM`), used whenever no model is given, e.g. by `VE(t)`.

The Earth's orbit is taken as circular, in the ecliptic, i.e. at zeroth order in its eccentricity [10, 11]:

$$\vec{v}_{ES}(t) = V_{ES}\left[\hat{\epsilon}_1 \cos\omega(t - t_{\mathrm{eq}}) + \hat{\epsilon}_2 \sin\omega(t - t_{\mathrm{eq}})\right],$$

- $\omega = 2\pi/\mathrm{yr}$ is the orbital angular frequency.
- $V_{ES} = a\omega \approx 29.79$ km/s, with a = 1 au.
- t_{eq} is the March equinox.
- $\hat{\epsilon}_1$ is the direction of the Earth's velocity, and $\hat{\epsilon}_2$ the Earth → Sun direction, at t_{eq}.

Both basis vectors are expressed in Galactic-Rectangular coordinates, to first order in the slow precession of the equinoxes (measured from `J2000`), and evaluated at the epoch of t_{eq} [10, Eqs. (13-15)].

TODO: Include an elliptical model

[10] gives $\vec{v}_{ES}$ to first order in the eccentricity, but Analyses relying on e.g. an annual modulation might need a more-accurate elliptic model of the Earth's orbit.

The Earth's galactic velocity is then

$$\vec{v}_E(t) = \vec{v}_\odot + \vec{v}_{ES}(t),$$

whose speed peaks at T_0 (about June 1st), when $\vec{v}_{ES}$ is most closely aligned with $\vec{v}_\odot$. Times may be given as a `DateTime`, a `Date`, a `DayOfYear`, or a `Unitful` time, which is read as the time elapsed since T_0:

`StandardHaloModelsPP.TimeLike` – Type.

```
TimeLike = Union{DateTime, Date, DayOfYear, Unitful.Time}
```

source

`StandardHaloModelsPP.DayOfYear` – Type.

```
DayOfYear(d::Real[, year::Integer = Dates.year(T0)])
```

Day `d` of `year` as a time argument: `d = 1` is January 1st, 00:00 UTC, and fractional days are allowed, so `DayOfYear(153.5)` is noon on June 1st 2020.

A plain number is deliberately *not* accepted as a time; instead use a `Unitful` time such as `100u"d"` to mean the time elapsed since `T0`.

source

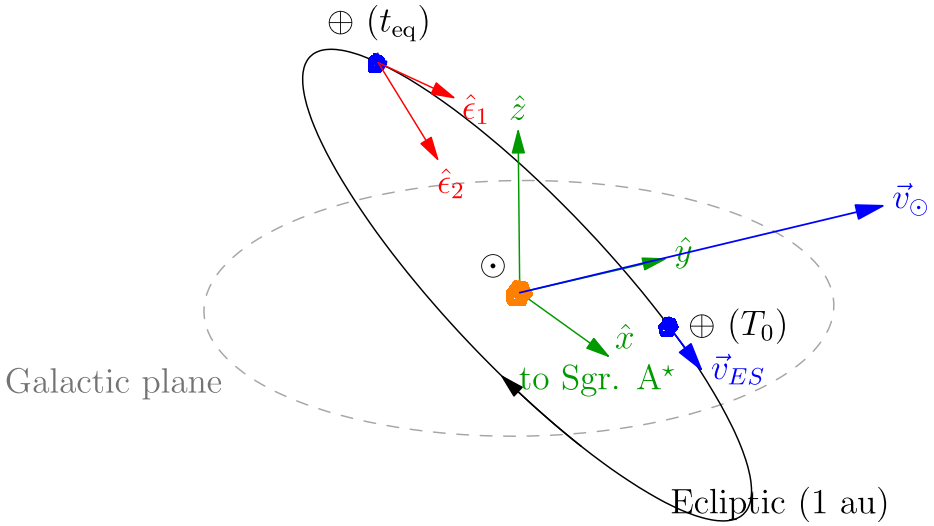


Figure 5.2: The Earth's orbit (the ecliptic, at 1 au) in Galactic-Rectangular coordinates, with the Earth at the March equinox $t_{\rm eq}$ and at T_0. The velocities $\vec{v}_\odot$ and $\vec{v}_{ES}$ are drawn to a common scale.

(Fig. eph.asy source)

`StandardHaloModelsPP.T0` – Constant.

June 1st: the time at which the Earth's orbital velocity $\vec{v}_{ES}$ is most closely aligned with the Sun's $\vec{v}_\odot$, so the Earth's galactic speed peaks, for the reference-value VLSR. Value: 2020-06-01T03:50:00

T0 is the time origin for every model (e.g. of `Unitful` times), so it does not move with a model's LSR speed. For the provided preset models the alignment stays within a day of it (half a day earlier for SHMlegacy).

source

`StandardHaloModelsPP.J2000` – Constant.

The J2000.0 epoch, 2000-01-01 12:00 (TT, taken as UTC at this precision). Value: 2000-01-01T12:00:00

source

`StandardHaloModelsPP.vES` – Function.

Velocity of the Earth in the Solar frame as a function of time, for a circular orbit:

$$\mathbf{v}_{ES}(t) = V_{ES}\left[\hat{\epsilon}_1 \cos\omega(t - t_{\rm eq}) + \hat{\epsilon}_2 \sin\omega(t - t_{\rm eq})\right]$$

the zeroth order in the eccentricity of [11, Eq. (B.7)] and [10, Eq. (21)].

source

`StandardHaloModelsPP.vE` – Function.

Velocity of the Earth in the galactic frame as a function of time, with the reference-value VLSR. See also `vE(m, t)` for a model's own.

source

```
vE(m::HaloModel, t::TimeLike)
VE(m::HaloModel, t::TimeLike)
```

The Earth's galactic velocity and speed at `t`, for model `m`: boosted with the model's own LSR speed `m.VLSR` (by default its v_0) rather than the reference-value VLSR.

If `m` does not contain uncertainties then ephemeris uncertainties are not added to the result (for performance reasons), otherwise uncertainties will be included.

Unqualified `vE(t)`,`VE(t)` use the reference-value VLSR, and always carry uncertainties.

source

`StandardHaloModelsPP.VE` – Function.

Speed of the Earth in the galactic frame as a function of time, with the reference-value VLSR. See also `VE(m, t)` for a model's own.

source

`StandardHaloModelsPP.vE` – Method.

```
vE(m::HaloModel, t::TimeLike)
VE(m::HaloModel, t::TimeLike)
```

The Earth's galactic velocity and speed at `t`, for model `m`: boosted with the model's own LSR speed `m.VLSR` (by default its v_0) rather than the reference-value VLSR.

If `m` does not contain uncertainties then ephemeris uncertainties are not added to the result (for performance reasons), otherwise uncertainties will be included.

Unqualified `vE(t)`,`VE(t)` use the reference-value VLSR, and always carry uncertainties.

source

The annual mean of the Earth's speed, and the time at which it is reached, are also available, though not exported (as `StandardHaloModelsPP.VEavg` and `StandardHaloModelsPP.Tavg`):

`StandardHaloModelsPP.VEavg` – Constant.

Annual mean of VE, with the reference-value VLSR.

Value: 252.1 ± 2.5 km s^-1

source

`StandardHaloModelsPP.Tavg` – Constant.

Time after T0 at which VE rises through its annual mean VEavg, so `VE(Tavg)` ≈ VEavg. About March 1st.

VE also falls through its mean roughly half a year later, around September 1st.

Value: 273.03012243637755 d

source

Chapter 6

Lab-Frame Distributions

The lab-frame distributions, transforming the velocity argument with the model's ephemeris (`vE(m, t)`):

`StandardHaloModelsPP.fχlab` – Method.

```
fχlab([m::HaloModel = SHM,] [t::TimeLike = T0,] v::AbstractVector{<:Unitful.Velocity})
fχlab(m::HaloModel) -> ([t,] v) -> fχlab(m, t, v)
fχlab(m::HaloModel, t::TimeLike) -> v -> fχlab(m, t, v)
```

Lab-frame `fχ`: `fχ(m, v + vE(m, t))`. For `fχlab(m)` see "Repeated evaluation" under `fχ`; `fχlab(m, t)` also fixes the time, and so the Earth's velocity.

source

`StandardHaloModelsPP.fRlab` – Method.

```
fRlab([m::HaloModel = SHM,] [t::TimeLike = T0,] v::AbstractVector{<:Unitful.Velocity})
fRlab(m::HaloModel) -> ([t,] v) -> fRlab(m, t, v)
fRlab(m::HaloModel, t::TimeLike) -> v -> fRlab(m, t, v)
```

Lab-frame `fR`: `fR(m, v + vE(m, t))`. For `fRlab(m)` see "Repeated evaluation" under `fχ`; `fRlab(m, t)` also fixes the time, and so the Earth's velocity.

source

To evaluate either at many velocities, see Performance Tips.

TODO: Sidereal Motion of the Lab

Analyses involving daily (sidereal) modulation and directional signals need the lab's motion and orientation with the Earth's rotation, as a function of location on its surface and the time of day.

Part III

Velocity Distribution Integrals

`StandardHaloModelsPP.jl` provides a variety of utilities for integrals of the lab-frame distribution; its marginals, the mean speed and mean reciprocal speed above a threshold, and integrals of general kernels against it. They are computed in closed form where one exists, and otherwise with Integrals.jl. See also the Performance Tips.

Uncertainties

By default these integrals evaluate the central values of the model's parameters, even for a model carrying `Measurements` such as `SHMpp`. Pass `uncertainty = true` to propagate them instead. That is as much as 10-130x slower, since Measurements.jl tracks derivatives through every evaluation of the integrand.

TODO: Faster uncertainty propagation options

Dual numbers (e.g. ForwardDiff.jl) would give the result and its gradient in one integration, with the escape-boundary terms still added separately. Monte Carlo over an `EnsembleProblem` of sampled models (e.g. MonteCarloMeasurements.jl) would give the full distribution, where first order is poor (e.g. v_{min} near `vmaxlab`).

Chapter 7

Marginalized Distributions

The lab-frame distribution marginalized over $v_z \equiv v_\phi$ (the planar distribution) and over solid angle (the speed distribution). For $\eta = 0$ models the speed distribution has a closed form [16, Eq. (I.55)], used by default:

`StandardHaloModelsPP.fχplanar` – Method.

```
fχplanar([m::HaloModel = SHM,] [t::TimeLike = T0,] vr::Unitful.Velocity, vθ::Unitful.Velocity;
↪  uncertainty = false, alg = QuadGKJL(), reltol = 1e-6, abstol = 0, kwargs...)
```

The galactic-planar velocity distribution, marginalized over $v_z \sim v_\phi$:

$$f^{\text{planar}}_{\chi,\text{lab}}(v_r, v_\theta) = \int f_{\chi,\text{lab}}(\mathbf{v})\, dv_z$$

`abstol` defaults to `0`, a purely relative tolerance. It may be a `Quantity` in the units of the result; a plain number is taken in the integrand's internal (unit-stripped) units. Further `kwargs` are passed to Integrals.jl's `solve`.

By default the central values of `m`'s parameters (and of the ephemeris) are integrated. Pass `uncertainty = true` to propagate a `Measurement` model's uncertainties (typically 10-130x slower due to derivative tracking/propagation).

See also `fχspeed` for the marginalization over solid angle.

source

`StandardHaloModelsPP.fχspeed` – Method.

```
fχspeed(m::HaloModel, t::TimeLike; uncertainty = false, analytic = true, alg = HCubatureJL(),
↪  reltol = 1e-4, abstol = 0, kwargs...) -> v -> fχspeed(m, t, v)
fχspeed([m::HaloModel = SHM,] [t::TimeLike = T0,] v::Unitful.Velocity; uncertainty = false,
↪  analytic = true, alg = HCubatureJL(), reltol = 1e-4, abstol = 0, kwargs...)
```

Marginalized speed distribution, integrated over solid angle:

$$f^{\text{speed}}_{\chi,\text{lab}}(v) = v^2 \int f_{\chi,\text{lab}}(\mathbf{v})\, d\Omega$$

normalized so $\int f\, dv = 1$.

When $\eta = 0$ and the Earth is inside the escape sphere ($V_E < v_{\text{esc}}$, true of all preset models) an exact analytic form [16, Sec. I.4] is used; pass `analytic=false` to force numerical integration. `abstol` defaults

to `0`, a purely relative tolerance. It may be a `Quantity` in the units of the result; a plain number is taken in the integrand's internal (unit-stripped) units. Further `kwargs` are passed to Integrals.jl's `solve`.

By default the central values of `m`'s parameters (and of the ephemeris) are integrated. Pass `uncertainty = true` to propagate a `Measurement` model's uncertainties (typically 10-130x slower due to derivative tracking/propagation).

The two-argument form `fχspeed(m, t)` returns the distribution at `t` as a callable `v -> fχspeed(m, t, v)`, with the lab frame, normalizations and the analytic-or-numerical choice fixed. Use it to evaluate repeatedly, e.g. inside your own integrand. It does not warn on non-convergence per evaluation (check caller's outer integral instead).

See also `fχplanar` for the marginalization over v_z.

source

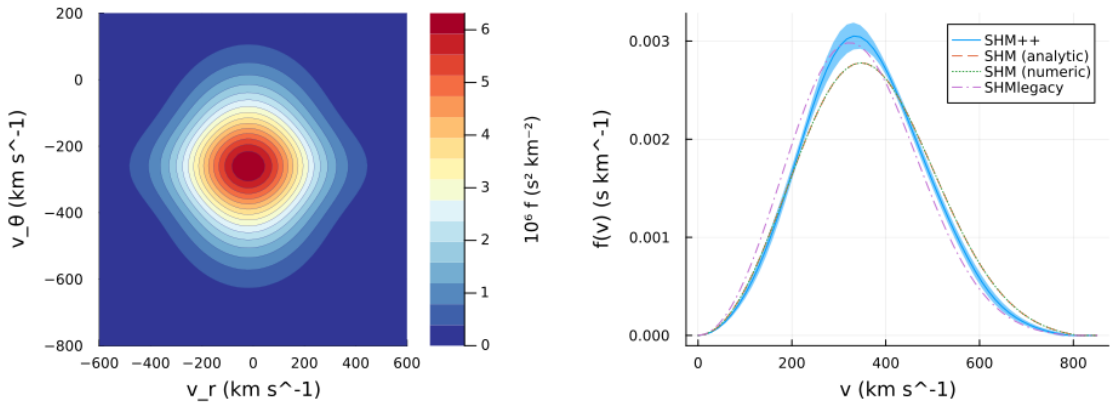
These recreate plots akin to [5, Fig. 1], at `T0`. The paper's SHM is `SHMlegacy` here, and its SHM++ band spans η, while the ribbon here propagates all of SHMpp's uncertainties:

```
using Plots, NaturallyUnitful, StandardHaloModelsPP
using Measurements: value, uncertainty

# Marginalized planar velocity distribution
vrrange = LinRange(-600, 600, 72)u"km/s"
vθrange = LinRange(-800, 200, 60)u"km/s"
# (the colorbar needs plain numbers, so scale and unit-strip by-hand)
z = [1e6 * ustrip(u"s^2/km^2", fχplanar(SHMpp, vr, vθ)) for vθ in vθrange, vr in vrrange]
p1 = contourf(vrrange, vθrange, z; color = cgrad(:RdYlBu, rev = true), lw = 0.2,
              xlabel = "v_r", ylabel = "v_θ", colorbar_title = "10⁶ f (s² km⁻²)")

# Speed distribution, with its uncertainty as a ribbon (Unitful values give axis units)
vrange = LinRange(0, 850, 60)u"km/s"
y = fχspeed.(SHMpp, vrange; uncertainty = true)
p2 = plot(vrange, value.(y); ribbon = uncertainty.(y), label = "SHM++",
          xlabel = "v", ylabel = "f(v)")
plot!(p2, vrange, fχspeed.(SHM, vrange); label = "SHM (analytic)", ls = :dash)
plot!(p2, vrange, fχspeed.(SHM, vrange; analytic = false); label = "SHM (numeric)", ls = :dot)
plot!(p2, vrange, fχspeed.(SHMlegacy, vrange); label = "SHMlegacy", ls = :dashdot)

plot(p1, p2; layout = 2, size = (950, 360), margin = 6Plots.mm)
```



Chapter 8

Mean Speed and Reciprocal

The mean reciprocal speed g (elsewhere often $\eta(v_{\text{min}})$) and the mean speed h above a threshold v_{min} are the halo integrals of velocity-independent and velocity-dependent scattering rates [16]. For $\eta = 0$ models they have closed forms [16, Eqs. (I.60, I.63)], used by default:

`StandardHaloModelsPP.g` – Method.

```
g([m::HaloModel = SHM,] [t::TimeLike = T0,] vmin::Unitful.Velocity; uncertainty = false, analytic
↪  = true, alg = HCubatureJL(), reltol = 1e-4, abstol = 0, kwargs...)
g(m::HaloModel, t::TimeLike; uncertainty = false, analytic = true, alg = HCubatureJL(), reltol =
↪  1e-4, abstol = 0, kwargs...) -> vmin -> g(m, t, vmin)
```

The mean reciprocal speed $g(v_{\text{min}}, t) = \int_{v>v_{\text{min}}} \frac{f_{\chi,\text{lab}}(\mathbf{v})}{v}\, d^3v$.

When $\eta = 0$ and the Earth is inside the escape sphere ($V_E < v_{\text{esc}}$, true of all preset models) an exact analytic form [16, Sec. I.4] is used; pass `analytic=false` to force numerical integration. `abstol` defaults to `0`, a purely relative tolerance. It may be a `Quantity` in the units of the result; a plain number is taken in the integrand's internal (unit-stripped) units. Further `kwargs` are passed to Integrals.jl's `solve`.

By default the central values of `m`'s parameters (and of the ephemeris) are integrated. Pass `uncertainty = true` to propagate a `Measurement` model's uncertainties (typically 10-130x slower due to derivative tracking/propagation).

The two-argument form `g(m, t)` returns the integral at `t` as a callable `vmin -> g(m, t, vmin)` (taking the same keywords), with the analytic-or-numerical choice fixed and, on the analytic path, the Earth's speed and the normalization computed once. Use it to evaluate many v_{min} at one time, e.g. across a recoil-energy spectrum.

source

`StandardHaloModelsPP.h` – Method.

```
h([m::HaloModel = SHM,] [t::TimeLike = T0,] vmin::Unitful.Velocity; uncertainty = false, analytic
↪  = true, alg = HCubatureJL(), reltol = 1e-4, abstol = 0, kwargs...)
h(m::HaloModel, t::TimeLike; uncertainty = false, analytic = true, alg = HCubatureJL(), reltol =
↪  1e-4, abstol = 0, kwargs...) -> vmin -> h(m, t, vmin)
```

The mean speed $h(v_{\text{min}}, t) = \int_{v>v_{\text{min}}} v\, f_{\chi,\text{lab}}(\mathbf{v})\, d^3v$.

When $\eta = 0$ and the Earth is inside the escape sphere ($V_E < v_{\text{esc}}$, true of all preset models) an exact analytic form [16, Sec. I.4] is used; pass `analytic=false` to force numerical integration. `abstol` defaults to `0`, a purely relative tolerance. It may be a `Quantity` in the units of the result; a plain number is taken in the integrand's internal (unit-stripped) units. Further `kwargs` are passed to Integrals.jl's `solve`.

By default the central values of `m`'s parameters (and of the ephemeris) are integrated. Pass `uncertainty = true` to propagate a `Measurement` model's uncertainties (typically 10-130x slower due to derivative tracking/propagation).

The two-argument form `h(m, t)` returns the integral at `t` as a callable `vmin -> h(m, t, vmin)` (taking the same keywords), with the analytic-or-numerical choice fixed and, on the analytic path, the Earth's speed and the normalization computed once. Use it to evaluate many v_{min} at one time, e.g. across a recoil-energy spectrum.

source

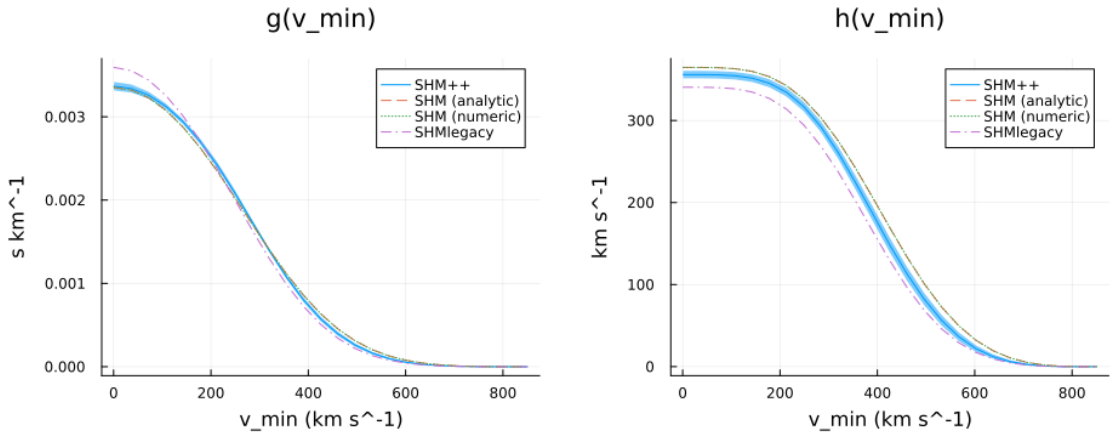
For the presets:

```
using Plots, NaturallyUnitful, StandardHaloModelsPP
using Measurements: value, uncertainty

vrange = LinRange(1, 850, 25)u"km/s"
y = g.(SHMpp, vrange; uncertainty = true)
p1 = plot(vrange, value.(y); ribbon = uncertainty.(y), label = "SHM++",
          title = "g(v_min)", xlabel = "v_min")
plot!(p1, vrange, g.(SHM, vrange); label = "SHM (analytic)", ls = :dash)
plot!(p1, vrange, g.(SHM, vrange; analytic = false); label = "SHM (numeric)", ls = :dot)
plot!(p1, vrange, g.(SHMlegacy, vrange); label = "SHMlegacy", ls = :dashdot)

y = h.(SHMpp, vrange; uncertainty = true)
p2 = plot(vrange, value.(y); ribbon = uncertainty.(y), label = "SHM++",
          title = "h(v_min)", xlabel = "v_min")
plot!(p2, vrange, h.(SHM, vrange); label = "SHM (analytic)", ls = :dash)
plot!(p2, vrange, h.(SHM, vrange; analytic = false); label = "SHM (numeric)", ls = :dot)
plot!(p2, vrange, h.(SHMlegacy, vrange); label = "SHMlegacy", ls = :dashdot)

plot(p1, p2; layout = 2, size = (950, 360), margin = 6Plots.mm)
```



Chapter 9

Arbitrary Convolutions

`velint` integrates a kernel against the full 3-D lab-frame distribution, and `speedint` a scalar kernel against the speed distribution. A kernel may be any callable taking `(v, p)` or just `(v)`. Both return the integral itself; for control over the algorithm and error estimate, or for sweeps, use the Integration Problems they are built on. For which to use, see Performance Tips.

`StandardHaloModelsPP.velint` – Method.

```
velint([m::HaloModel = SHM,] [t::TimeLike = T0,] kk; uncertainty = false, vmin = 0u"km/s", p =
↪  (), alg = HCubatureJL(), reltol = 1e-4, abstol = 0.0, kwargs...)
```

Arbitrary convolution $\int_{v>v_{\min}} f_{\chi,\mathrm{lab}}(\mathbf{v})\, k(\mathbf{v},p)\, d^3v$ of a kernel `kk(v, p)` (or `kk(v)`) with the lab-frame velocity distribution: `solve(HaloIntegralProblem(m, t, kk; ...), alg; ...).u`. See `HaloIntegralProblem` for the problem itself, e.g. to choose an algorithm, inspect the error estimate or sweep `t`.

`abstol` defaults to `0`, a purely relative tolerance. It may be a `Quantity` in the units of the result; a plain number is taken in the integrand's internal (unit-stripped) units. Further `kwargs` are passed to Integrals.jl's `solve`.

By default the central values of `m`'s parameters (and of the ephemeris) are integrated. Pass `uncertainty = true` to propagate a `Measurement` model's uncertainties (typically 10-130x slower due to derivative tracking/propagation).

See also `speedint`, which convolves against the marginalized speed distribution.

source

`StandardHaloModelsPP.speedint` – Method.

```
speedint([m::HaloModel = SHM,] [t::TimeLike = T0,] kk; uncertainty = false, vmin = 0u"km/s", vmax
↪  = nothing, p = (), alg = QuadGKJL(), reltol = 1e-6, abstol = 0.0, analytic = true, inneralg =
↪  HCubatureJL(), innerreltol = reltol / 10, innerabstol = abstol isa Quantity ? 0.0 : abstol /
↪  10, kwargs...)
```

Convolution $\int k(v,p)\, f^{\mathrm{speed}}_{\chi,\mathrm{lab}}(v)\, dv$ of a scalar kernel `kk(v, p)` (or `kk(v)`) with the marginalized speed distribution: `solve(HaloIntegralProblem(m, t, kk; measure = :speed, ...), alg; ...).u`. See `HaloIntegralProblem` for the problem itself.

`abstol` defaults to `0`, a purely relative tolerance, as for `velint`. It may be a `Quantity` in the units of the result; a plain number is interpreted as being in the integrand's internal stripped units. The default `innerabstol` is a tenth of a plain-number `abstol`, and `0` for a `Quantity` one, since the inner integral has different units; with the default `abstol` it is `0`, so the inner quadrature is controlled by `innerreltol` alone.

Pass `innerabstol` explicitly, as a number or a `Quantity` in the units of `fχspeed`, to override. Further kwargs are passed to the outer Integrals.jl `solve`.

By default the central values of `m`'s parameters (and of the ephemeris) are integrated. Pass `uncertainty = true` to propagate a `Measurement` model's uncertainties (typically 10-130x slower due to derivative tracking/propagation).

See also `velint`, which convolves against the full 3-D velocity distribution.

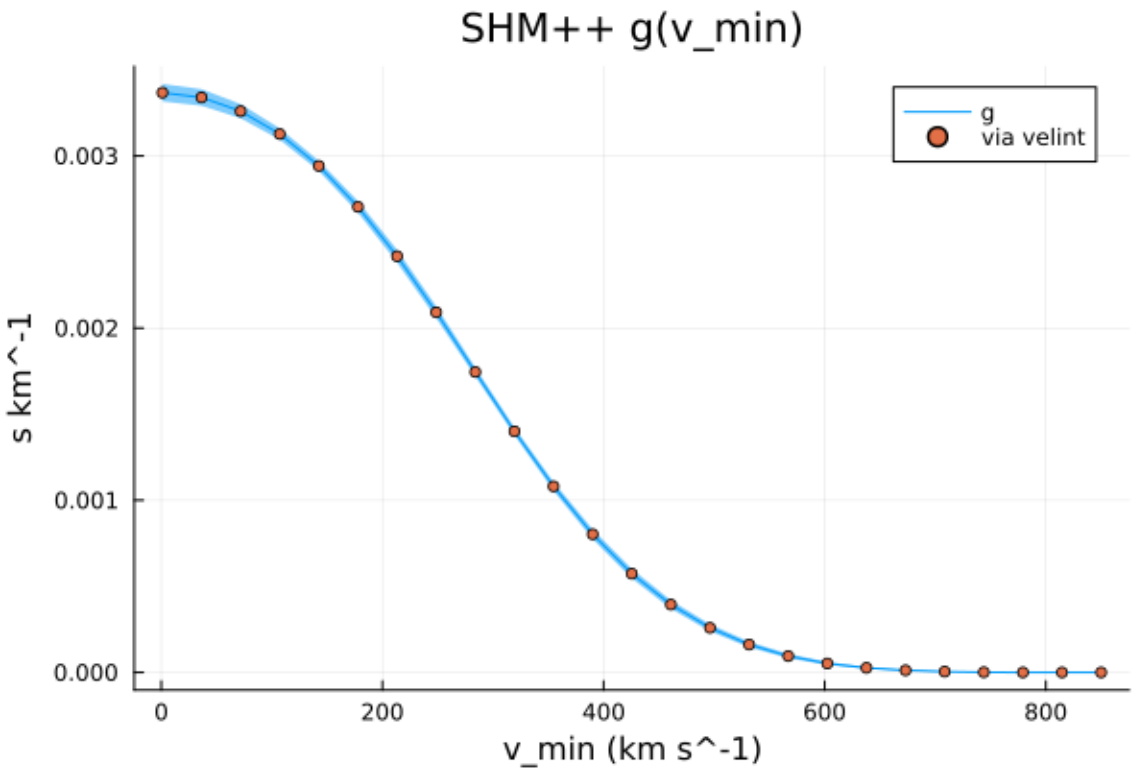
source

For example, g is `velint` with the kernel $1/|\mathbf{v}|$. The two agree, here for the SHM++, where g is itself numerical:

```
using Plots, NaturallyUnitful, LinearAlgebra, StandardHaloModelsPP
using Measurements: value, uncertainty

gg(m, vmin) = velint(m, v -> 1 / norm(v); vmin)

vrange = LinRange(1, 850, 25)u"km/s"
y, yy = g.(SHMpp, vrange; uncertainty = true), gg.(SHMpp, vrange)
p = plot(vrange, value.(y); ribbon = uncertainty.(y), label = "g", title = "SHM++ g(v_min)",
         xlabel = "v_min")
scatter!(p, vrange, yy; label = "via velint", markersize = 3)
```



Chapter 10

Integration Problems

`velint` and `speedint` wrap a `HaloIntegralProblem`: an Integrals.jl `IntegralProblem` for the integral, with units handled for you. Use the problem directly to choose an algorithm, inspect the error estimate or return code, or `remake` it with a parameter changed. `solve`, `remake` and the SciML ensemble types (`EnsembleProblem`, `EnsembleSerial`, `EnsembleThreads`, `EnsembleDistributed`) are re-exported.

`StandardHaloModelsPP.HaloIntegralProblem` – Type.

```
HaloIntegralProblem([m::HaloModel = SHM,] [t::TimeLike = T0,] kk; measure = :velocity,
↪  uncertainty = false, kwargs...)
```

An integral problem with a kernel `kk` against the lab-frame velocity distribution of `m` at time `t`, set up for Integrals.jl. The kernel `kk` may be any callable (a function, or a callable object) taking `(v, p)` or just `(v)`: with the default `p = ()` it is called as `kk(v)` if it has a method for that (so `norm` or `inv ∘ norm` work), otherwise as `kk(v, ())`; with a `p`, as `kk(v, p)` if it has a method for that, otherwise as `kk(v)`. `solve` it with any Integrals.jl algorithm:

```
using LinearAlgebra: norm
using Integrals: HCubatureJL
using Dates: DateTime, Day

prob = HaloIntegralProblem(SHM, v -> 1 / norm(v); vmin = 300u"km/s")
sol = solve(prob)
sol = solve(prob, HCubatureJL(); reltol = 1e-6, maxiters = 10^6)
sol.u                                    # integral solution with units
remake(prob; t = T0 + Day(90))           # integral at another time
```

Measures

- `measure = :velocity`: $\int_{v>v_{\min}} f_{\chi,\mathrm{lab}}(\mathbf{v})\, k(\mathbf{v},p)\, d^3v$, with `kk(v::AbstractVector, p)`. Keywords: `vmin`, `p`. The problem is 3D over rescaled spherical coordinates $(s,w,\alpha)\in[0,1]\times[0,1]\times[0,2\pi]$, polar about the Earth's velocity, spanning exactly the distribution's support above $v_{\min}$: the escape-speed boundary is the domain boundary, and near $v_{\max}$ the domain shrinks onto the small cap of directions that still reach it; default algorithm `HCubatureJL()`, `reltol = 1e-4`, `abstol = 0`. See `velint`.
- `measure = :speed`: $\int_{v_{\min}}^{v_{\max}} k(v,p)\, f^{\mathrm{speed}}_{\chi,\mathrm{lab}}(v)\, dv$, with `kk(v::Unitful.Velocity, p)`. Keywords: `vmin`, `vmax` (default $v_{\mathrm{esc}}+V_E$), `p`, `analytic`, and the inner angular quadrature's `inneralg`, `innerreltol` (`1e-7`), `innerabstol` (`0`). The problem is 1D over v; the inner integral, `fχspeed`, is fixed at construction, and is analytic when $\eta=0$ and $V_E<v_{\mathrm{esc}}$. Default algorithm `QuadGKJL()`, `reltol = 1e-6`, `abstol = 0`. See `speedint`.

Solving

`solve(prob[, alg]; reltol, abstol, kwargs...)` returns a `HaloIntegralSolution`; further kwargs go to Integrals.jl's `solve`. `abstol` may be a `Quantity` in the units of the result; a plain number is taken in the integrand's internal (unit-stripped) units. `remake(prob; kwargs...)` rebuilds the problem with any constructor argument changed (`m`, `t`, `kernel`, `measure`, `uncertainty`, or the measure's keywords), at a few percent of the cost of a solve.

Sweeps

A sweep is a SciML `EnsembleProblem` whose `prob_func` remakes the problem per point; it returns an `EnsembleSolution` of `HaloIntegralSolution`s, and runs on threads with `EnsembleThreads()` (start Julia with `-t auto`):

```
ts = [DateTime(2020, 1, 1) + Day(d) for d in 0:364]
ens = EnsembleProblem(prob; prob_func = (p, ctx) -> remake(p; t = ts[ctx.sim_id]))
sols = solve(ens, nothing, EnsembleThreads(); trajectories = length(ts))
```

Pass `nothing` as the algorithm for the problem's default. Kernel parameters sweep the same way, e.g. `remake(p; p = Es[ctx.sim_id])` for a kernel `kk(v, p)`.

Uncertainties

By default only the central values of `m`'s parameters (and of the ephemeris) are integrated. `uncertainty = true` additionally propagates a `Measurement` model's uncertainties. That is typically 10-130x slower, and only works with pure-Julia algorithms (e.g. `QuadGKJL`, `HCubatureJL`): C-backed ones (Cuba, Cubature) need plain floating-point integrands. The defaults measure error on central values (`norm = value ∘ norm`); a user-supplied adaptive algorithm without that norm still works, but possibly more slowly.

source

`StandardHaloModelsPP.HaloIntegralSolution` – Type.

```
HaloIntegralSolution
```

The result of `solve`-ing a `HaloIntegralProblem`.

- `u`: the integral, with units.
- `resid`: the solver's error estimate, in the same units, or `nothing` if the algorithm does not provide one.
- `retcode`: the SciML return code; checked by `SciMLBase.successful_retcode(sol)`.
- `sol`: the underlying (unit-free) Integrals.jl solution.

source

10.1 Sweeps

A sweep over time, v_{min}, kernel parameters or models is a SciML `EnsembleProblem` whose `prob_func` remakes the problem for each point. Rebuilding costs about 3% of a solve, and the result is an `EnsembleSolution` of `HaloIntegralSolution`s. With `EnsembleThreads()` the points are solved in parallel (start Julia with e.g. `-t auto`):

```
using Dates
ts = [DateTime(2020, 1, 1) + Day(d) for d in 0:364]            # annual modulation
ens = EnsembleProblem(prob; prob_func = (p, ctx) -> remake(p; t = ts[ctx.sim_id]))
sols = solve(ens, nothing, EnsembleThreads(); trajectories = length(ts))
[s.u for s in sols]
```

Passing `nothing` as the algorithm uses the problem's default.

10.2 Building Your Own Integrals

To integrate against the halo in your own `IntegralProblem` (e.g. jointly with a recoil energy), use the distributions frozen at a time, which fix the lab frame and normalizations once and evaluate cheaply: `fχspeed(m, t)` for the speed distribution (analytic where possible, and quiet inside an integrand) and `fχlab(m, t)` for the full velocity distribution. `vmaxlab` gives the largest lab-frame speed with support, and `vE(m, t)` / `VE(m, t)` the model's Earth velocity and speed.

`StandardHaloModelsPP.vmaxlab` – Function.

```
vmaxlab([m::HaloModel = SHM,] [t::TimeLike = T0])
```

The largest lab-frame speed at which `m`'s distribution is nonzero at time `t`: $v_{\text{esc}} + V_E(t)$. For $v_{\text{esc}} = \infty$ (e.g. `SHMnoesc`) the escape speed is the effective cutoff $6v_0$ (beyond which the Maxwellian is below e^{-36}).

Like the pointwise distributions a model `m` with uncertainties returns a `Measurement` (with the ephemeris' uncertainties included, see `VE`); use `value(vmaxlab(m, t))` for a plain integration limit.

source

TODO: Vector space methods for anisotropic detectors

Provide basis projections (spherical harmonics × radial functions) of the distributions for the vector space methods of [17, 18], as implemented in VSDM [19]: rates become matrix products, and detector orientations rotations of the coefficients.

Part IV

Performance Tips

Chapter 11

Evaluating a Distribution at Many Points

Each call like `fχ(m, v)` first computes the model's normalizations, which for a model carrying `Measurements` costs several times the evaluation itself. To evaluate repeatedly, *freeze* the model once, and call the result:

```
f = fχ(SHMpp)           # normalizations computed once
f.(vs)                  # or f(v) in a loop, a closure, your own integrand, ...

flab = fχlab(SHMpp, t)  # also fixes the time, and so the Earth's velocity
flab(v)
```

The same works for `fR`, `fS`, `fRlab` and `fχlab` (`fχlab(m)` takes `([t,] v)`; `fχlab(m, t)` takes `v`).

Broadcasting over velocities with a single model does this for you: `fχ.(m, vs)`, `fχlab.(m, t, vs)` and the default-model `fχ.(vs)` all freeze the model once. For 1000 points of `SHMpp` this is about 7× faster than calling `fχ(m, v)` per point. Broadcasting over an array of models still evaluates each model separately, as usual.

Chapter 12

Uncertainties Cost Time

A model carrying `Measurements` (such as `SHMpp`) makes every evaluation propagate uncertainties: about 10x slower pointwise. When you don't need them, use its central-value model `value(m)` instead, which evaluates as fast as an exact one.

Integrals evaluate central values by default; `uncertainty = true` propagates the model's uncertainties, at roughly 10-130x the cost (most for `g` and `h`).

Chapter 13

Analytic Fast Paths

For a model without the anisotropic component ($\eta = 0$, i.e. every preset but `SHMpp`), with the Earth inside its escape sphere, `fχspeed`, `g` and `h` use exact closed forms [16, Sec. I.4], which take under 100 ns. Pass `analytic = false` to force the numerical integrals instead (e.g. to check them).

Chapter 14

Choosing Between `velint` and `speedint`

For a kernel that depends only on the speed $|\mathbf{v}|$, both `velint` (the 3-D velocity integral) and `speedint` (a 1-D integral over the speed distribution) apply:

- For $\eta = 0$ models, `speedint` uses the analytic speed distribution and is the faster: around 20 µs, against 0.04-0.2 ms for `velint`.
- For anisotropic models (e.g. `SHMpp`), every point of the speed distribution is itself an angular integral, and `velint` is faster (about 0.04-2 ms against 0.4-5 ms, both at a relative tolerance of 10^{-4}). Both get cheaper as v_{min} rises.

`speedint` defaults to a tighter `reltol` (10^{-6}) than `velint` (10^{-4}). For an anisotropic model comparing defaults is a misleading benchmark (a `speedint` takes around 50 ms). Pass `reltol` explicitly when comparing the two, or when the default precision isn't needed.

A kernel depending on the direction of $\mathbf{v}$ needs `velint`.

Chapter 15

Tolerances

Every integrator defaults to a purely relative tolerance: a `reltol` suited to its method, and `abstol = 0`. To set an absolute tolerance, give `abstol` as a `Quantity` in the units of the result. A plain number is taken in the integrand's internal units, so it only makes sense for integrals of order one: a small integral (e.g. a rate for a small cross-section, or g far into its tail) with a plain-number `abstol` can stop well short of its `reltol`.

Chapter 16

Many Integrals: Sweeps and Threads

To compute an integral at many times, $v_{\min}$ values or kernel parameters, solve an `EnsembleProblem` of `HaloIntegralProblem`s on threads, as described under Sweeps. Start Julia with threads (e.g. `julia -t auto`) for `EnsembleThreads()` to run in parallel.

To integrate against the distribution inside your own integrals, use the forms frozen at a time, `fχspeed(m, t)` and `fχlab(m, t)`, rather than calling `fχspeed(m, t, v)` or `fχlab(m, t, v)` at every point (see Building Your Own Integrals). Likewise, to evaluate `g` or `h` at many $v_{\min}$ at one time (e.g. across a recoil-energy spectrum), use `g(m, t)` and `h(m, t)`: on the analytic path they take around 25-40 ns per call, against 100-130 ns for `g(m, t, vmin)`.

Part V

Reference

Chapter 17

Reference

17.1 Bibliography

[1] J. D. Lewin and P. F. Smith. *Review of mathematics, numerical factors, and corrections for dark matter experiments based on elastic nuclear recoil*. Astropart. Phys. **6**, 87–112 (1996).

[2] D. Baxter and others. *Recommended conventions for reporting results from direct dark matter searches*. Eur. Phys. J. C **81**, 907 (2021), arXiv:2105.00599 [hep-ex].

[3] V. Belokurov, D. Erkal, N. W. Evans, S. E. Koposov and A. J. Deason. *Co-formation of the disc and the stellar halo★*. Monthly Notices of the Royal Astronomical Society **478**, 611–619 (2018), arXiv:1802.03414.

[4] L. Necib, M. Lisanti and V. Belokurov. *Inferred Evidence for Dark Matter Kinematic Substructure with SDSS–Gaia*. The Astrophysical Journal **874**, 3 (2019), arXiv:1807.02519.

[5] N. W. Evans, C. A. O'Hare and C. McCabe. *SHM++: Refinement of the standard halo model for dark matter searches in light of the Gaia Sausage*. Phys. Rev. D **99**, 023012 (2019), arXiv:1810.11468.

[6] N. W. Evans, C. A. O'Hare and C. McCabe. *SHM^{++}: A Refinement of the Standard Halo Model for Dark Matter Searches in Light of the Gaia Sausage* (2018), arXiv:1810.11468 [astro-ph.GA].

[7] M. C. Smith and others. *The RAVE Survey: Constraining the Local Galactic Escape Speed*. Mon. Not. Roy. Astron. Soc. **379**, 755–772 (2007), arXiv:astro-ph/0611671.

[8] R. Schönrich, J. Binney and W. Dehnen. *Local kinematics and the local standard of rest*. Monthly Notices of the Royal Astronomical Society **403**, 1829–1833 (2010), arXiv:0912.3693 [astro-ph.GA].

[9] R. Abuter and others. *Improved GRAVITY astrometric accuracy from modeling optical aberrations*. Astron. Astrophys. **647**, A59 (2021), arXiv:2101.12098 [astro-ph.GA].

[10] C. McCabe. *The Earth's velocity for direct detection experiments*. Journal of Cosmology and Astroparticle Physics **2014**, 027 (2014), arXiv:1312.1355.

[11] S. K. Lee, M. Lisanti and B. R. Safdi. *Dark-Matter Harmonics Beyond Annual Modulation*. JCAP **11**, 033 (2013), arXiv:1307.5323 [hep-ph].

[12] J. Binney and M. Merrifield. *Galactic Astronomy*. *Princeton Series in Astrophysics* (Princeton University Press, Princeton, NJ, 1998).

[13] R. Abuter and others. *Detection of the gravitational redshift in the orbit of the star S2 near the Galactic centre massive black hole*. Astron. Astrophys. **615**, L15 (2018), arXiv:1807.09409 [astro-ph.GA].

[14] J. Bland-Hawthorn and O. Gerhard. *The Galaxy in Context: Structural, Kinematic, and Integrated Properties*. Ann. Rev. Astron. Astrophys. **54**, 529–596 (2016), arXiv:1602.07702 [astro-ph.GA].

[15] M. Jurić and others. *The Milky Way Tomography with SDSS. 1. Stellar Number Density Distribution*. Astrophys. J. **673**, 864–914 (2008), arXiv:astro-ph/0510520.

[16] E. Del Nobile. *The Theory of Direct Dark Matter Detection: A Guide to Computations*. *Lecture Notes in Physics* (Springer, Cham, 2022), arXiv:2104.12785 [hep-ph]. Originally released as Appendiciario: A hands-on manual on the theory of direct Dark Matter detection.

[17] B. Lillard. *Vector Space Integration for Dark Matter Scattering* (2025), arXiv:2310.01480 [hep-ph].

[18] B. Lillard. *Wavelet-Harmonic Integration Methods* (2025), arXiv:2310.01483 [hep-ph].

[19] B. Lillard and A. Radick. *Vector Spaces for Dark Matter (VSDM): Fast Direct Detection Calculations with Python and Julia* (2025), arXiv:2502.17547 [hep-ph].

17.2 API

- `StandardHaloModelsPP.J2000`
- `StandardHaloModelsPP.R0`
- `StandardHaloModelsPP.RES`
- `StandardHaloModelsPP.SHM`
- `StandardHaloModelsPP.SHMerr`
- `StandardHaloModelsPP.SHMlegacy`
- `StandardHaloModelsPP.SHMnoesc`
- `StandardHaloModelsPP.SHMpp`
- `StandardHaloModelsPP.T0`
- `StandardHaloModelsPP.Tavg`
- `StandardHaloModelsPP.Teq`
- `StandardHaloModelsPP.VES`
- `StandardHaloModelsPP.VEavg`
- `StandardHaloModelsPP.VLSR`
- `StandardHaloModelsPP.VSUN`
- `StandardHaloModelsPP.d0`
- `StandardHaloModelsPP.vLSR`
- `StandardHaloModelsPP.vPEC`
- `StandardHaloModelsPP.vPECc`
- `StandardHaloModelsPP.vSUN`
- `StandardHaloModelsPP.z0`
- `StandardHaloModelsPP.η0`
- `StandardHaloModelsPP.θ0`

- StandardHaloModelsPP.ωES
- StandardHaloModelsPP.ϕ0
- StandardHaloModelsPP.ϵ1
- StandardHaloModelsPP.ϵ2
- StandardHaloModelsPP.DayOfYear
- StandardHaloModelsPP.HaloIntegralProblem
- StandardHaloModelsPP.HaloIntegralSolution
- StandardHaloModelsPP.HaloModel
- StandardHaloModelsPP.HaloModel
- StandardHaloModelsPP.PreparedHalo
- StandardHaloModelsPP.TimeLike
- Measurements.value
- StandardHaloModelsPP.VE
- StandardHaloModelsPP._capangle
- StandardHaloModelsPP._jcenturies
- StandardHaloModelsPP._radialsupport
- StandardHaloModelsPP._speedproblem
- StandardHaloModelsPP._vSUN
- StandardHaloModelsPP._velocityproblem
- StandardHaloModelsPP.fR
- StandardHaloModelsPP.fRlab
- StandardHaloModelsPP.fS
- StandardHaloModelsPP.fχ
- StandardHaloModelsPP.fχlab
- StandardHaloModelsPP.fχplanar
- StandardHaloModelsPP.fχspeed
- StandardHaloModelsPP.g
- StandardHaloModelsPP.h
- StandardHaloModelsPP.prepare
- StandardHaloModelsPP.speedint
- StandardHaloModelsPP.vE
- StandardHaloModelsPP.vE
- StandardHaloModelsPP.vES
- StandardHaloModelsPP.velint
- StandardHaloModelsPP.vmaxlab

Exported

`StandardHaloModelsPP.J2000` – Constant.

The J2000.0 epoch, 2000-01-01 12:00 (TT, taken as UTC at this precision). Value: 2000-01-01T12:00:00

source

`StandardHaloModelsPP.R0` – Constant.

Radial (in-galactic-plane/cylindrical) distance of the sun from the Galactic Center (Sgr. A*).

Value: $R_0 = d_0 \cos\eta_0 = 8.275 \pm 0.034$ kpc

source

`StandardHaloModelsPP.SHM` – Constant.

The default `HaloModel` for the SHM, following the recommended conventions for reporting direct-detection results in D. Baxter et al., *Eur. Phys. J. C* **81**, 907 (2021) (arXiv)

SHM		
Local DM density	ρ_0	0.3 GeV cm^{-3}
Circular rotation speed	v_0	238 km s^{-1}
Escape speed	v_{esc}	544 km s^{-1}
Velocity distribution	$f_R(\mathbf{v})$	Gaussian

(additional SHM++ parameters are zeroed.) [2, Table 1]

`v₀`, and so the LSR speed of the lab-frame boost, is the reference ephemeris' `VLSR` (its central value). For the pre-2021 `v₀ = 220` km/s convention see `SHMlegacy`.

source

`StandardHaloModelsPP.SHMerr` – Constant.

`SHM` with measurement uncertainties on its two characteristic speeds *included*, for sensitivity studies of Halo Model conventions and uncertainties.

SHMerr		
Local DM density	ρ_0	0.3 GeV cm^{-3} (fixed)
Circular rotation speed	v_0	238.0 ± 1.5 km s^{-1}
Escape speed	v_{esc}	544 ± 33 km s^{-1}

- `v₀`: using uncertainty quoted but not used in [2, Sec. 3.1.5].
- `vₑ`: using the RAVE 90% confidence interval $498 < v_{\text{esc}} < 608$ km/s of [7] (median 544 km/s, agreeing with the `SHM` convention), symmetrized and converted to 1σ: $(608-498)/(2\times1.645) \approx 33$ km/s.
- `ρ₀` is a fixed value rather than a measurement with uncertainty, as it is not a directly measured quantity; set another value (usually in the range 0.2-0.6 GeV/cm³) by creating your own `HaloModel` from this one via `HaloModel`.
- $v_{\text{LSR}} = v_0$ as `Measurements.jl` objects so the dispersion and the boost are correctly correlated.

Note

[2, Sec. 3.1] recommends against attaching uncertainties to the SHM parameters when reporting results: they "do not accurately reflect the uncertainties in the dark matter halo, nor do they represent likelihood distributions that can be meaningfully profiled over". `SHM` therefore stays exact (and is the default model); use `SHMerr` for sensitivity studies, but not for reporting limits. Propagating Measurements uncertainties also has a significant performance impact on integrals.

source

`StandardHaloModelsPP.SHMlegacy` – Constant.

A historical SHM, with `v₀ = 220` km/s with lab-frame boosts commensurate (otherwise as `SHM`), for comparison with results that predate [2]; see e.g. [1] and [6, Table 1].

source

`StandardHaloModelsPP.SHMnoesc` – Constant.

An untruncated variant of `SHM`: `vₑ = Inf`, so the Maxwellian has no escape-speed cutoff and normalizes to unity over all velocities.

source

`StandardHaloModelsPP.SHMpp` – Constant.

The default `HaloModel` for the SHM++.

SHM++		
Local DM density	ρ_0	0.55 ± 0.17 GeV cm^{-3}
Circular rotation speed	v_0	233 ± 3 km s^{-1}
Escape speed	$v_{\rm esc}$	528 ± 25 km s^{-1}
Sausage anisotropy	β	0.9 ± 0.05
Sausage fraction	η	0.2 ± 0.1
Velocity distribution	$f_\chi(\mathbf{v})$	Gaussian+Sausage

[6, Table 1]

As in the defining SHM++ paper, the lab-frame boost uses $v_{\rm LSR} = v_0 = 233 \pm 3$ km/s, with the boost and the dispersion correlated. Create your own modified copy if you want a SHM++ halo model with Baxter [2] parameters instead.

You may want to strip Measurement uncertainties via `Measurements.value(m)` if you are not interested in sensitivity questions wrt. the Gaia sausage and just want to use this as a halo model.

source

`StandardHaloModelsPP.T0` – Constant.

June 1st: the time at which the Earth's orbital velocity $\vec{v}_{ES}$ is most closely aligned with the Sun's $\vec{v}_\odot$, so the Earth's galactic speed peaks, for the reference-value `VLSR`. Value: 2020-06-01T03:50:00

`T0` is the time origin for every model (e.g. of `Unitful` times), so it does not move with a model's LSR speed. For the provided preset models the alignment stays within a day of it (half a day earlier for `SHMlegacy`).

source

`StandardHaloModelsPP.TimeLike` – Type.

```
TimeLike = Union{DateTime, Date, DayOfYear, Unitful.Time}
```

source

StandardHaloModelsPP.VLSR – Constant.

Reference circular speed of the Local Standard of Rest (LSR), used by ephemeris functions when no model is given.

Value: 238.0 ± 1.5 km s^-1

D. Baxter and others, Eur. Phys. J. C 81, 907 (2021); Sec. 3.1.5.

Note

A HaloModel carries its own LSR speed m.VLSR, which by default is its circular speed v_0. The model-aware vE(m, t) and every lab-frame distribution and integral use that parameter when a model is available, not this constant. It equals this one for SHM and SHMerr, but is 233 km/s for SHMpp and 220 km/s for SHMlegacy, as in their defining papers.

The derived reference constants vSUN, T0, VEavg and Tavg use this value whatever the model. T0 in particular is a fixed date.

source

StandardHaloModelsPP.VSUN – Constant.

Speed of the sun in the galactic frame. A good ballpark typical DM velocity ignoring Earth ephemeris.

source

StandardHaloModelsPP.d0 – Constant.

Distance of the sun from the Galactic Center (Sgr. A*).

Value: $d_0 = 8.275 \pm 0.034$ kpc.

R. Aboter and others (GRAVITY collaboration). Astron. Astrophys. 647, A59 (2021): $8275 \pm 9_{\text{stat}} \pm 33_{\text{sys}}$ pc, combined in quadrature. This is the distance underlying VLSR in [2]; it supersedes the 8.122 ± 0.031 kpc of [13] used in the defining SHM++ paper.

source

StandardHaloModelsPP.vLSR – Constant.

Circular velocity (vector) of the Local Standard of Rest (LSR) in galactic rectangular coordinates.

Value: $v_{\text{LSR}} = (v_x, v_y, v_z) =$ (0.0 ± 0.0 km s^-1, 238.0 ± 1.5 km s^-1, 0.0 ± 0.0 km s^-1)

source

StandardHaloModelsPP.vPEC – Constant.

Peculiar velocity of the sun in the Local Standard of Rest (LSR) in galactic rectangular coordinates.

Value: $v_{\text{PEC}} = (U, V, W) =$ (11.1 ± 1.5 km s^-1, 12.2 ± 2.0 km s^-1, 7.3 ± 1.0 km s^-1)

R. Schönrich, J. Binney, W. Dehnen, Mon. Not. Roy. Astron. Soc. 403, 1829 (2010), with the (statistical and systematic) uncertainties as adopted by the SHM++.

source

`StandardHaloModelsPP.vSUN` – Constant.

Velocity of the sun in the Galactic frame, using the reference-value VLSR. The halo's Galactocentric-Spherical velocity components are taken along the Galactic-Rectangular axes at the Sun [5].

Value: $v_\odot = (v_x, v_y, v_z) = (v_r, v_\theta, v_\phi) =$ (11.1 ± 1.5 km s^-1, 250.2 ± 2.5 km s^-1, 7.3 ± 1.0 km s^-1)

source

`StandardHaloModelsPP.z0` – Constant.

Normal distance of the sun above the Galactic Plane.

Value: $z_0 =$ 25.0 ± 5.0 pc

J. Bland-Hawthorn and O. Gerhard, Ann. Rev. Astron. Astrophys. 54, 529 (2016), adopting the SDSS estimate of M. Jurić and others (SDSS collaboration), Astrophys. J. 673, 864 (2008).

source

`StandardHaloModelsPP.η0` – Constant.

Altitude (angle) of the sun above the Galactic Plane.

Value: $\eta_0 = \sin^{-1} z_0/d_0 =$ 0.00302 ± 0.0006 (radians)

source

`StandardHaloModelsPP.θ0` – Constant.

Azimuthal angle of the solar system in Galactocentric Spherical Coordinates.

Value: $\theta_0 =$ 0.0

(Zero by definition.)

source

`StandardHaloModelsPP.ϕ0` – Constant.

Polar angle of the sun from the galactic north zenith.

Value: $\phi_0 = \frac{\pi}{2} - \eta_0 =$ 1.56778 ± 0.0006 (radians)

source

`StandardHaloModelsPP.DayOfYear` – Type.

```
DayOfYear(d::Real[, year::Integer = Dates.year(T0)])
```

Day `d` of `year` as a time argument: `d = 1` is January 1st, 00:00 UTC, and fractional days are allowed, so `DayOfYear(153.5)` is noon on June 1st 2020.

A plain number is deliberately *not* accepted as a time; instead use a `Unitful` time such as `100u"d"` to mean the time elapsed since `T0`.

source

`StandardHaloModelsPP.HaloIntegralProblem` – Type.

```
HaloIntegralProblem([m::HaloModel = SHM,] [t::TimeLike = T0,] kk; measure = :velocity,
↪  uncertainty = false, kwargs...)
```

An integral problem with a kernel `kk` against the lab-frame velocity distribution of `m` at time `t`, set up for Integrals.jl. The kernel `kk` may be any callable (a function, or a callable object) taking `(v, p)` or just `(v)`: with the default `p = ()` it is called as `kk(v)` if it has a method for that (so `norm` or `inv ∘ norm` work), otherwise as `kk(v, ())`; with a `p`, as `kk(v, p)` if it has a method for that, otherwise as `kk(v)`. `solve` it with any Integrals.jl algorithm:

```
using LinearAlgebra: norm
using Integrals: HCubatureJL
using Dates: DateTime, Day

prob = HaloIntegralProblem(SHM, v -> 1 / norm(v); vmin = 300u"km/s")
sol = solve(prob)
sol = solve(prob, HCubatureJL(); reltol = 1e-6, maxiters = 10^6)
sol.u                                    # integral solution with units
remake(prob; t = T0 + Day(90))           # integral at another time
```

Measures

- `measure = :velocity`: $\int_{v>v_{\min}} f_{\chi,\mathrm{lab}}(\mathbf{v})\, k(\mathbf{v},p)\, d^3v$, with `kk(v::AbstractVector, p)`. Keywords: `vmin`, `p`. The problem is 3D over rescaled spherical coordinates $(s,w,\alpha)\in[0,1]\times[0,1]\times[0,2\pi]$, polar about the Earth's velocity, spanning exactly the distribution's support above $v_{\min}$: the escape-speed boundary is the domain boundary, and near $v_{\max}$ the domain shrinks onto the small cap of directions that still reach it; default algorithm `HCubatureJL()`, `reltol = 1e-4`, `abstol = 0`. See `velint`.
- `measure = :speed`: $\int_{v_{\min}}^{v_{\max}} k(v,p)\, f^{\mathrm{speed}}_{\chi,\mathrm{lab}}(v)\, dv$, with `kk(v::Unitful.Velocity, p)`. Keywords: `vmin`, `vmax` (default $v_{\mathrm{esc}}+V_E$), `p`, `analytic`, and the inner angular quadrature's `inneralg`, `innerreltol` (`1e-7`), `innerabstol` (`0`). The problem is 1D over v; the inner integral, `fχspeed`, is fixed at construction, and is analytic when $\eta=0$ and $V_E<v_{\mathrm{esc}}$. Default algorithm `QuadGKJL()`, `reltol = 1e-6`, `abstol = 0`. See `speedint`.

Solving

`solve(prob[, alg]; reltol, abstol, kwargs...)` returns a `HaloIntegralSolution`; further `kwargs` go to Integrals.jl's `solve`. `abstol` may be a `Quantity` in the units of the result; a plain number is taken in the integrand's internal (unit-stripped) units. `remake(prob; kwargs...)` rebuilds the problem with any constructor argument changed (`m`, `t`, `kernel`, `measure`, `uncertainty`, or the measure's keywords), at a few percent of the cost of a solve.

Sweeps

A sweep is a SciML `EnsembleProblem` whose `prob_func` remakes the problem per point; it returns an `EnsembleSolution` of `HaloIntegralSolution`s, and runs on threads with `EnsembleThreads()` (start Julia with `-t auto`):

```
ts = [DateTime(2020, 1, 1) + Day(d) for d in 0:364]
ens = EnsembleProblem(prob; prob_func = (p, ctx) -> remake(p; t = ts[ctx.sim_id]))
sols = solve(ens, nothing, EnsembleThreads(); trajectories = length(ts))
```

Pass `nothing` as the algorithm for the problem's default. Kernel parameters sweep the same way, e.g. `remake(p; p = Es[ctx.sim_id])` for a kernel `kk(v, p)`.

Uncertainties

By default only the central values of `m`'s parameters (and of the ephemeris) are integrated. `uncertainty = true` additionally propagates a `Measurement` model's uncertainties. That is typically 10-130x slower, and

only works with pure-Julia algorithms (e.g. `QuadGKJL`, `HCubatureJL`): C-backed ones (Cuba, Cubature) need plain floating-point integrands. The defaults measure error on central values (`norm = value ∘ norm`); a user-supplied adaptive algorithm without that norm still works, but possibly more slowly.

source

`StandardHaloModelsPP.HaloIntegralSolution` – Type.

```
HaloIntegralSolution
```

The result of `solve`-ing a `HaloIntegralProblem`.

- `u`: the integral, with units.
- `resid`: the solver's error estimate, in the same units, or `nothing` if the algorithm does not provide one.
- `retcode`: the SciML return code; checked by `SciMLBase.successful_retcode(sol)`.
- `sol`: the underlying (unit-free) Integrals.jl solution.

source

`StandardHaloModelsPP.HaloModel` – Type.

```
HaloModel(ρ₀::Union{Unitful.Pressure,Unitful.Density}, v₀::Unitful.Velocity,
↪  vₑ::Unitful.Velocity, β::Real, η::Real[, VLSR::Unitful.Velocity])
```

An SHM++-parameterized halo model, per the definition in N. Wyn Evans, Ciaran A.J. O'Hare, Christopher McCabe, *Phys. Rev. D.* **99**, 023012 (2019) (arXiv).

- `localdensity`: ρ_0, local DM mass density: with mass in energy units (e.g. `u"GeV/cm^3"`, hence a `Unitful` pressure), or as a mass density (e.g. `UnitfulAstro`: `u"Msun/pc^3"`); units are stored as given.
- `circspeed`: v_0, circular speed of the flat asymptote of the galactic rotation curve, which sets the DM velocity dispersion $\sigma = v_0/\sqrt{2}$.
- `escspeed`: v_e, galactic escape speed.
- `anisotropy`: β, Gaia sausage anisotropy, $0 \le \beta < 1$.
- `sausagefrac`: η, Gaia sausage prominence, $0 \le \eta \le 1$.
- `lsrspeed`: VLSR, speed of the Local Standard of Rest, which sets the boost to the lab frame. By default (omitted, or `nothing`) it is v_0: the SHM's isothermal halo has a flat rotation curve with circular speed $v_0 = \sqrt{2}\sigma$, so the two coincide [5]. Give it to boost with a different speed than the dispersion implies. `m.VLSR` is always the speed in use, even when tied to v_0.

source

`StandardHaloModelsPP.HaloModel` – Method.

```
HaloModel(; ρ₀, v₀, vₑ, β = 0, η = 0, VLSR = nothing)
HaloModel(m::HaloModel; kwargs...)
```

Keyword forms of the `HaloModel` constructor. Each parameter may be named by its field name or a symbol shorthand (v_0 or `circspeed`, v_e or `escspeed`, ...).

Also a copying form which copies `m` with the given parameters changed, e.g. `HaloModel(SHMpp; v₀ = 220u"km/s")`. Unchanged `Measurement` parameters are carried over as the same objects to maintain correlations.

This also allows e.g. Accessors.jl to work automatically on symbol shorthands: `@set SHMpp.v₀ = 220u"km/s"`.

source

StandardHaloModelsPP.VE – Method.

Speed of the Earth in the galactic frame as a function of time, with the reference-value VLSR. See also `VE(m, t)` for a model's own.

source

StandardHaloModelsPP.fR – Method.

```
fR([m::HaloModel = SHM,] v::AbstractVector{<:Unitful.Velocity})
fR([m::HaloModel = SHM,] v::Unitful.Velocity)
fR(m::HaloModel) -> v -> fR(m, v)
```

Galactic-frame Maxwellian component of f_χ.

$$f_R(\mathbf{v}) = \frac{1}{(2\pi\sigma_v^2)^{3/2} N_{R,\text{esc}}} \exp\left(-\frac{|\mathbf{v}|^2}{2\sigma_v^2}\right) \Theta(v_\text{esc} - |\mathbf{v}|)$$

$$N_{R,\text{esc}} = \text{erf}\left(\frac{v_\text{esc}}{\sqrt{2}\sigma_v}\right) - \sqrt{\frac{2}{\pi}} \frac{v_\text{esc}}{\sigma_v} \exp\left(-\frac{v_\text{esc}^2}{2\sigma_v^2}\right)$$

with $\sigma_v = v_0/\sqrt{2}$. Isotropic, so v may be given as a scalar speed.

For `fR(m)` see "Repeated evaluation" under fχ.

source

StandardHaloModelsPP.fRlab – Method.

```
fRlab([m::HaloModel = SHM,] [t::TimeLike = T0,] v::AbstractVector{<:Unitful.Velocity})
fRlab(m::HaloModel) -> ([t,] v) -> fRlab(m, t, v)
fRlab(m::HaloModel, t::TimeLike) -> v -> fRlab(m, t, v)
```

Lab-frame fR: `fR(m, v + vE(m, t))`. For `fRlab(m)` see "Repeated evaluation" under fχ; `fRlab(m, t)` also fixes the time, and so the Earth's velocity.

source

StandardHaloModelsPP.fS – Method.

```
fS([m::HaloModel = SHM,] v::AbstractVector{<:Unitful.Velocity})
fS(m::HaloModel) -> v -> fS(m, v)
```

Galactic-frame anisotropic (Gaia sausage) component of f_χ.

$$f_S(\mathbf{v}) = \frac{1}{(2\pi)^{3/2}\sigma_r\sigma_\theta^2 N_{S,\text{esc}}} \exp\left(-\frac{v_r^2}{2\sigma_r^2} - \frac{v_\theta^2}{2\sigma_\theta^2} - \frac{v_\phi^2}{2\sigma_\phi^2}\right) \Theta(v_\text{esc} - |\mathbf{v}|)$$

$$N_{S,\text{esc}} = \text{erf}\left(\frac{v_\text{esc}}{\sqrt{2}\sigma_r}\right) - \sqrt{\frac{1-\beta}{\beta}} \exp\left(-\frac{v_\text{esc}^2}{2\sigma_\theta^2}\right) \text{erfi}\left(\frac{v_\text{esc}}{\sqrt{2}\sigma_r}\sqrt{\frac{\beta}{1-\beta}}\right)$$

Anisotropic, so v must be a vector; `v[1]` is the radial (sausage) axis v_r.

For `fS(m)` see "Repeated evaluation" under fχ.

source

StandardHaloModelsPP.fχ – Method.

```
fχ([m::HaloModel = SHM,] v::AbstractVector{<:Unitful.Velocity})
fχ(m::HaloModel) -> v -> fχ(m, v)
```

Galactic-frame SHM++ velocity distribution: a mixture of the round `fR` and anisotropic `fS` components.

$$f_\chi(\mathbf{v}) = (1-\eta) f_R(\mathbf{v}) + \eta f_S(\mathbf{v})$$

Repeated evaluation

Each `fχ(m, v)` call recomputes the model's normalizations (`erf`, `erfi` and `exp` of the parameters), which for a `Measurement` model costs several times the evaluation itself.

The one-argument form `f = fχ(m)` computes them once and returns a callable `f(v)`, to use in hot loops, closures or integrands. Broadcasting over velocities with a single model, as in `fχ.(m, vs)` or `fχ.(vs)`, does this automatically.

The same applies to `fR`, `fS`, `fRlab` and `fχlab`.

source

StandardHaloModelsPP.fχlab – Method.

```
fχlab([m::HaloModel = SHM,] [t::TimeLike = T0,] v::AbstractVector{<:Unitful.Velocity})
fχlab(m::HaloModel) -> ([t,] v) -> fχlab(m, t, v)
fχlab(m::HaloModel, t::TimeLike) -> v -> fχlab(m, t, v)
```

Lab-frame `fχ`: `fχ(m, v + vE(m, t))`. For `fχlab(m)` see "Repeated evaluation" under `fχ`; `fχlab(m, t)` also fixes the time, and so the Earth's velocity.

source

StandardHaloModelsPP.fχplanar – Method.

```
fχplanar([m::HaloModel = SHM,] [t::TimeLike = T0,] vr::Unitful.Velocity, v0::Unitful.Velocity;
↪  uncertainty = false, alg = QuadGKJL(), reltol = 1e-6, abstol = 0, kwargs...)
```

The galactic-planar velocity distribution, marginalized over $v_z \sim v_\phi$:

$$f^{\text{planar}}_{\chi,\text{lab}}(v_r, v_\theta) = \int f_{\chi,\text{lab}}(\mathbf{v})\, dv_z$$

`abstol` defaults to `0`, a purely relative tolerance. It may be a `Quantity` in the units of the result; a plain number is taken in the integrand's internal (unit-stripped) units. Further `kwargs` are passed to Integrals.jl's `solve`.

By default the central values of `m`'s parameters (and of the ephemeris) are integrated. Pass `uncertainty = true` to propagate a `Measurement` model's uncertainties (typically 10-130x slower due to derivative tracking/propagation).

See also `fχspeed` for the marginalization over solid angle.

source

StandardHaloModelsPP.fχspeed – Method.

```
fχspeed(m::HaloModel, t::TimeLike; uncertainty = false, analytic = true, alg = HCubatureJL(),
↪  reltol = 1e-4, abstol = 0, kwargs...) -> v -> fχspeed(m, t, v)
fχspeed([m::HaloModel = SHM,] [t::TimeLike = T0,] v::Unitful.Velocity; uncertainty = false,
↪  analytic = true, alg = HCubatureJL(), reltol = 1e-4, abstol = 0, kwargs...)
```

Marginalized speed distribution, integrated over solid angle:

$$f_{\chi,\mathrm{lab}}^{\mathrm{speed}}(v) = v^2 \int f_{\chi,\mathrm{lab}}(\mathbf{v})\, d\Omega$$

normalized so $\int f\, dv = 1$.

When $\eta = 0$ and the Earth is inside the escape sphere ($V_E < v_{\mathrm{esc}}$, true of all preset models) an exact analytic form [16, Sec. I.4] is used; pass `analytic=false` to force numerical integration. `abstol` defaults to `0`, a purely relative tolerance. It may be a `Quantity` in the units of the result; a plain number is taken in the integrand's internal (unit-stripped) units. Further `kwargs` are passed to Integrals.jl's `solve`.

By default the central values of `m`'s parameters (and of the ephemeris) are integrated. Pass `uncertainty = true` to propagate a `Measurement` model's uncertainties (typically 10-130x slower due to derivative tracking/propagation).

The two-argument form `fχspeed(m, t)` returns the distribution at `t` as a callable `v -> fχspeed(m, t, v)`, with the lab frame, normalizations and the analytic-or-numerical choice fixed. Use it to evaluate repeatedly, e.g. inside your own integrand. It does not warn on non-convergence per evaluation (check caller's outer integral instead).

See also `fχplanar` for the marginalization over v_z.

source

`StandardHaloModelsPP.g` – Method.

```
g([m::HaloModel = SHM,] [t::TimeLike = T0,] vmin::Unitful.Velocity; uncertainty = false, analytic
↪  = true, alg = HCubatureJL(), reltol = 1e-4, abstol = 0, kwargs...)
g(m::HaloModel, t::TimeLike; uncertainty = false, analytic = true, alg = HCubatureJL(), reltol =
↪  1e-4, abstol = 0, kwargs...) -> vmin -> g(m, t, vmin)
```

The mean reciprocal speed $g(v_{\mathrm{min}}, t) = \int_{v>v_{\mathrm{min}}} \frac{f_{\chi,\mathrm{lab}}(\mathbf{v})}{v}\, d^3v$.

When $\eta = 0$ and the Earth is inside the escape sphere ($V_E < v_{\mathrm{esc}}$, true of all preset models) an exact analytic form [16, Sec. I.4] is used; pass `analytic=false` to force numerical integration. `abstol` defaults to `0`, a purely relative tolerance. It may be a `Quantity` in the units of the result; a plain number is taken in the integrand's internal (unit-stripped) units. Further `kwargs` are passed to Integrals.jl's `solve`.

By default the central values of `m`'s parameters (and of the ephemeris) are integrated. Pass `uncertainty = true` to propagate a `Measurement` model's uncertainties (typically 10-130x slower due to derivative tracking/propagation).

The two-argument form `g(m, t)` returns the integral at `t` as a callable `vmin -> g(m, t, vmin)` (taking the same keywords), with the analytic-or-numerical choice fixed and, on the analytic path, the Earth's speed and the normalization computed once. Use it to evaluate many v_{min} at one time, e.g. across a recoil-energy spectrum.

source

`StandardHaloModelsPP.h` – Method.

```
h([m::HaloModel = SHM,] [t::TimeLike = T0,] vmin::Unitful.Velocity; uncertainty = false, analytic
↪  = true, alg = HCubatureJL(), reltol = 1e-4, abstol = 0, kwargs...)
h(m::HaloModel, t::TimeLike; uncertainty = false, analytic = true, alg = HCubatureJL(), reltol =
↪  1e-4, abstol = 0, kwargs...) -> vmin -> h(m, t, vmin)
```

The mean speed $h(v_{\mathrm{min}}, t) = \int_{v>v_{\mathrm{min}}} v\, f_{\chi,\mathrm{lab}}(\mathbf{v})\, d^3v$.

When $\eta = 0$ and the Earth is inside the escape sphere ($V_E < v_{\text{esc}}$, true of all preset models) an exact analytic form [16, Sec. I.4] is used; pass `analytic=false` to force numerical integration. `abstol` defaults to `0`, a purely relative tolerance. It may be a `Quantity` in the units of the result; a plain number is taken in the integrand's internal (unit-stripped) units. Further `kwargs` are passed to Integrals.jl's `solve`.

By default the central values of `m`'s parameters (and of the ephemeris) are integrated. Pass `uncertainty = true` to propagate a `Measurement` model's uncertainties (typically 10-130x slower due to derivative tracking/propagation).

The two-argument form `h(m, t)` returns the integral at `t` as a callable `vmin -> h(m, t, vmin)` (taking the same keywords), with the analytic-or-numerical choice fixed and, on the analytic path, the Earth's speed and the normalization computed once. Use it to evaluate many v_{min} at one time, e.g. across a recoil-energy spectrum.

source

`StandardHaloModelsPP.speedint` – Method.

```
speedint([m::HaloModel = SHM,] [t::TimeLike = T0,] kk; uncertainty = false, vmin = 0u"km/s", vmax
↪  = nothing, p = (), alg = QuadGKJL(), reltol = 1e-6, abstol = 0.0, analytic = true, inneralg =
↪  HCubatureJL(), innerreltol = reltol / 10, innerabstol = abstol isa Quantity ? 0.0 : abstol /
↪  10, kwargs...)
```

Convolution $\int k(v,p)\, f^{\text{speed}}_{\chi,\text{lab}}(v)\, dv$ of a scalar kernel `kk(v, p)` (or `kk(v)`) with the marginalized speed distribution: `solve(HaloIntegralProblem(m, t, kk; measure = :speed, ...), alg; ...).u`. See `HaloIntegralProblem` for the problem itself.

`abstol` defaults to `0`, a purely relative tolerance, as for `velint`. It may be a `Quantity` in the units of the result; a plain number is interpreted as being in the integrand's internal stripped units. The default `innerabstol` is a tenth of a plain-number `abstol`, and `0` for a `Quantity` one, since the inner integral has different units; with the default `abstol` it is `0`, so the inner quadrature is controlled by `innerreltol` alone. Pass `innerabstol` explicitly, as a number or a `Quantity` in the units of `fχspeed`, to override. Further `kwargs` are passed to the outer Integrals.jl `solve`.

By default the central values of `m`'s parameters (and of the ephemeris) are integrated. Pass `uncertainty = true` to propagate a `Measurement` model's uncertainties (typically 10-130x slower due to derivative tracking/propagation).

See also `velint`, which convolves against the full 3-D velocity distribution.

source

`StandardHaloModelsPP.vE` – Method.

Velocity of the Earth in the galactic frame as a function of time, with the reference-value `VLSR`. See also `vE(m, t)` for a model's own.

source

`StandardHaloModelsPP.vE` – Method.

```
vE(m::HaloModel, t::TimeLike)
VE(m::HaloModel, t::TimeLike)
```

The Earth's galactic velocity and speed at `t`, for model `m`: boosted with the model's own LSR speed `m.VLSR` (by default its v_0) rather than the reference-value `VLSR`.

If `m` does not contain uncertainties then ephemeris uncertainties are not added to the result (for performance reasons), otherwise uncertainties will be included.

Unqualified `vE(t)`,`VE(t)` use the reference-value `VLSR`, and always carry uncertainties.

source

`StandardHaloModelsPP.vES` – Method.

Velocity of the Earth in the Solar frame as a function of time, for a circular orbit:

$$\mathbf{v}_{ES}(t) = V_{ES}\left[\hat{\epsilon}_1 \cos\omega(t - t_{\mathrm{eq}}) + \hat{\epsilon}_2 \sin\omega(t - t_{\mathrm{eq}})\right]$$

the zeroth order in the eccentricity of [11, Eq. (B.7)] and [10, Eq. (21)].

source

`StandardHaloModelsPP.velint` – Method.

```
velint([m::HaloModel = SHM,] [t::TimeLike = T0,] kk; uncertainty = false, vmin = 0u"km/s", p =
↪  (), alg = HCubatureJL(), reltol = 1e-4, abstol = 0.0, kwargs...)
```

Arbitrary convolution $\int_{v>v_{\mathrm{min}}} f_{\chi,\mathrm{lab}}(\mathbf{v})\, k(\mathbf{v}, p)\, d^3v$ of a kernel `kk(v, p)` (or `kk(v)`) with the lab-frame velocity distribution: `solve(HaloIntegralProblem(m, t, kk; ...), alg; ...).u`. See `HaloIntegralProblem` for the problem itself, e.g. to choose an algorithm, inspect the error estimate or sweep `t`.

`abstol` defaults to `0`, a purely relative tolerance. It may be a `Quantity` in the units of the result; a plain number is taken in the integrand's internal (unit-stripped) units. Further `kwargs` are passed to Integrals.jl's `solve`.

By default the central values of `m`'s parameters (and of the ephemeris) are integrated. Pass `uncertainty = true` to propagate a `Measurement` model's uncertainties (typically 10-130x slower due to derivative tracking/propagation).

See also `speedint`, which convolves against the marginalized speed distribution.

source

`StandardHaloModelsPP.vmaxlab` – Method.

```
vmaxlab([m::HaloModel = SHM,] [t::TimeLike = T0])
```

The largest lab-frame speed at which `m`'s distribution is nonzero at time `t`: $v_{\mathrm{esc}} + V_E(t)$. For $v_{\mathrm{esc}} = \infty$ (e.g. `SHMnoesc`) the escape speed is the effective cutoff $6v_0$ (beyond which the Maxwellian is below e^{-36}).

Like the pointwise distributions a model `m` with uncertainties returns a `Measurement` (with the ephemeris' uncertainties included, see `VE`); use `value(vmaxlab(m, t))` for a plain integration limit.

source

Not exported

Unexported names, reachable under the module prefix (e.g. `StandardHaloModelsPP.VEavg`, or `Measurements.value` for the central-value model of a `HaloModel`), and internals documented for development.

`StandardHaloModelsPP.RES` – Constant.

Radius of Earth's orbit. Value: 1.49598e8 km

source

`StandardHaloModelsPP.Tavg` – Constant.

Time after T0 at which VE rises through its annual mean VEavg, so VE(Tavg) ≈ VEavg. About March 1st.

VE also falls through its mean roughly half a year later, around September 1st.

Value: 273.03012243637755 d

source

StandardHaloModelsPP.Teq – Constant.

March 20th, 2020 Equinox DateTime. Value: 2020-03-20T03:50:00

source

StandardHaloModelsPP.VES – Constant.

Tangential speed of the Earth in the Solar frame, $a\omega$, for a circular orbit. cf. $\langle u_E \rangle = 29.79$ km/s of [10, Eq. (12)].

Value: 29.785280109496657 km s^-1

source

StandardHaloModelsPP.VEavg – Constant.

Annual mean of VE, with the reference-value VLSR.

Value: 252.1 ± 2.5 km s^-1

source

StandardHaloModelsPP.vPECc – Constant.

Central-value solar peculiar velocity, for exact-model evaluation. See isexact.

source

StandardHaloModelsPP.ωES – Constant.

Angular frequency of the Earth's Solar orbit (rad/yr). Value: 6.283185307179586 yr^-1

source

StandardHaloModelsPP.ϵ1 – Constant.

Direction of the Earth's velocity at the March equinox Teq (equivalently the Sun → Earth direction at the June solstice), in galactic coordinates.

$\hat{\epsilon}_1 = \hat{\epsilon}_y(T)$ of [10, Eq. (14)], at the epoch of Teq; cf. its 2013 value in [11, Eq. (B.4)].

Value: [0.9940870341189444, 0.10859627785439957, 0.00465120030971937]

source

StandardHaloModelsPP.ϵ2 – Constant.

Earth → Sun direction at the March equinox Teq (equivalently the direction of the Earth's velocity at the June solstice), in galactic coordinates.

$\hat{\epsilon}_2 = -\hat{\epsilon}_x(T)$ of [10, Eq. (13)], at the epoch of Teq; cf. its 2013 value in [11, Eq. (B.4)].

Value: [-0.04997741430831242, 0.4946525909922808, -0.8676663125294437]

source

StandardHaloModelsPP.PreparedHalo – Type.

```
PreparedHalo
```

Internal: callable cache of the HaloModel normalizations for hoisting out of integrands. p(v) evaluates f_χ. Users can get the same effect through the frozen forms fχ(m), fR(m), ... (see fχ) without depending on this type.

source

Measurements.value – Method.

```
Measurements.value(m::HaloModel)
```

Copy m with every Measurement parameter replaced by its central value.

source

StandardHaloModelsPP._capangle – Method.

```
_capangle(Q, VEn, vmin) -> γ0
```

Polar angle from $\hat{V}$ beyond which a lab-frame direction n̂ has support at some speed v ≥ vmin, i.e. |v n̂ + V| < v_e; all of the integrand's support lies in γ ∈ [γ0, π].

Q = v_e^2 - |V|², VEn = |V|, vmin the kinematic lower cutoff. Along n̂ at angle γ the condition is cos γ < (Q - v²)/(2v|V|), largest at v = max(vmin, √(-Q)).

source

StandardHaloModelsPP._jcenturies – Method.

Julian centuries from J2000 to t: the epoch of date T of [10, Eq. (15)].

source

StandardHaloModelsPP._radialsupport – Method.

```
_radialsupport(Q, V, n, vmin) -> (lo, Δ)
```

Find the radial speed range with nonzero support from the halo.

V Earth galactic velocity, n lab-frame DM velocity unit vector, Q = v_e^2 - |V|², vmin the kinematic lower cutoff.

Returns (lo, Δ), lower bound and width of the range of speeds with support along n. (when Δ==0 lo is moved oneunit away from zero to avoid coordinate singularity).

source

StandardHaloModelsPP._speedproblem – Method.

```
_speedproblem(m::HaloModel, t, kk; vmin = 0u"km/s", vmax = nothing, p = (), kwargs...)
```

Construct the 1D speed integrand over v ∈ [vmin, vmax], in the units of vmin.

Returns the unit-free problem and the unit of its integral.

source

StandardHaloModelsPP._vSUN – Method.

The Sun's galactic velocity for model `m`: the model's LSR speed `m.VLSR` along $\hat{y}$, plus the peculiar velocity, which propagates its uncertainties only for a model that carries uncertainties.

source

`StandardHaloModelsPP._velocityproblem` – Method.

```
_velocityproblem(m::HaloModel, t, kk; vmin = 0u"km/s", p = ())
```

Construct the 3D velocity integrand over (s, w, α) ∈ [0,1] × [0,1] × [0,2π].

Kernel kk receives `(vxyz, vv, p)` with `vv = norm(vxyz)`.

Returns the unit-free problem and the unit of its integral. As in `_fχspeed`, directions n̂ are polar about V̂, at angle γ, and restricted to the cap γ ∈ [γ0, π] holding all the support above `vmin` (see `_capangle`), rescaled onto w ∈ [0,1]; α is the full azimuth, since the kernel need not be symmetric. The radial coordinate is rescaled onto s ∈ [0,1] spanning the support [lo, lo + Δ] along n̂. So the domain is the support, and adaptive cubature can't miss even a small cap near `vmaxlab`.

The support along n̂ shrinks to nothing at γ0, where the integrand vanishes, so the central-value bound γ0 needs no first-order boundary term for `Measurement` models.

source

`StandardHaloModelsPP.prepare` – Method.

```
prepare(m::HaloModel) -> PreparedHalo
```

Internal: hoist the normalizations out of hot loops. Reuse when evaluating repeatedly.

source